
gbemu

A Game Boy Emulator in C

Reference Manual

Konstantinos Despoinidis (KDesp73)
gbemu 0.1.0
*MIT licensed**

October 2026

Abstract

gbemu is a Game Boy and Game Boy Color emulator written in portable C99. The emulator core is a frontend-agnostic library: it knows nothing about windows, canvases or sound cards. Everything the core cannot do by itself — presenting a frame, reading a game pad, pushing audio samples — is delegated to a small vtable that a host program fills in. The same core therefore drives an SDL3 desktop build, a terminal front end that draws with ANSI half-blocks, a headless build used by the test harness, a WebAssembly build that runs in a browser, and an ESP32 firmware image.

This manual documents the whole system: how to build it, how to run it, how the memory map, the cartridge mappers, the timer, the PPU and the APU are implemented, how save files and save states are laid out on disk, how to embed the library in your own program, and how the test suites are driven. Where the implementation knowingly deviates from hardware, the deviation is stated rather than hidden.

Contents

1	Introduction	6
1.1	What gbemu is	6
1.2	Design goals	6
1.3	Feature summary	6
1.4	Accuracy and known limitations	7
1.5	Repository layout	8
1.6	Conventions used in this manual	8
1.7	License and authorship	9
2	Building gbemu	10
2.1	Prerequisites	10
2.2	Build targets	10

* The full text of the MIT license ships with the source tree in the file `LICENSE`.

2.3	Build types	11
2.4	Choosing a front end at build time	11
2.5	Build outputs	12
2.6	The WebAssembly target	12
2.7	The ESP32 target	13
2.8	Building this manual	13
3	Running the emulator	15
3.1	Command line	15
3.2	Environment variables	15
3.3	Controls	15
3.4	Hotkeys	16
3.5	Battery saves and save states	16
3.6	Shutting down	16
4	Architecture	17
4.1	Layers	17
4.2	Module map	17
4.3	The front-end vtable	18
4.4	The execution model	19
4.5	Time bases	20
4.6	State objects and ownership	20
4.7	Symbol prefixing and <code>\GBEMU_STRIP_PREFIX</code>	21
5	The CPU	22
5.1	Register file	22
5.2	Flags	23
5.3	Register-index helpers	23
5.4	Instruction dispatch	23
5.5	Generated opcode tables	25
5.6	<code>\CB</code> -prefixed instructions	25
5.7	Instruction timing	26
5.8	Interrupts	27
5.9	<code>\EI</code> , <code>\HALT</code> , the <code>HALT</code> bug and <code>\STOP</code>	28

6	The memory bus	29
6.1	Address decoding	29
6.2	I/O dispatch	29
6.3	Loading a ROM	30
6.4	Sanitising the interrupt vectors	30
6.5	The joypad register	31
6.6	Serial output	31
6.7	OAM DMA	31
6.8	\IF, \IE and the interrupt flags	32
6.9	The CGB speed switch	32
7	Memory bank controllers	33
7.1	MBC1	33
7.2	MBC2	34
7.3	MBC3	35
7.4	MBC5	35
7.5	Cartridge RAM sizing	36
8	The timer and divider	37
8.1	The divider	37
8.2	The \TIMA overflow delay	37
8.3	Register behaviour	38
9	The PPU	39
9.1	Registers	39
9.2	Mode state machine	39
9.3	Turning the LCD on and off	40
9.4	Rendering a scanline	40
9.5	Palettes	41
9.6	What the PPU does not do	42
10	The APU	43
10.1	Registers	43
10.2	Power	43
10.3	The frame sequencer	44
10.4	Frequency timers	44
10.5	Volume envelopes	44
10.6	Sweep	45
10.7	Length counters	45

10.8	Triggers and DAC	45
10.9	Mixing and output	45
10.10	Power-on defaults	46
10.11	What the APU does not do	46
11	Persistence	47
11.1	Battery saves	47
11.2	Save states	47
11.3	Validation	48
11.4	When things are written	49
12	Front ends	50
12.1	The SDL front end	50
12.2	The terminal front end	50
12.3	The headless front end	51
12.4	The WebAssembly front end	51
12.5	The ESP32 front end	52
13	Embedding the core	54
13.1	The minimal session	54
13.2	Running without a loop	54
13.3	Hotkeys and save states	55
13.4	Audio consumer	55
13.5	Linking	55
13.6	The WebAssembly host	56
14	Testing	57
14.1	The suites	57
14.2	Verdicts	57
14.3	Running the harness	58
14.4	Report files	58
14.5	Reading a failure	59
14.6	What the suites do not cover	59
15	Extending the emulator	60
15.1	Adding an instruction	60
15.2	Adding a component	60
15.3	Adding a front end	61
15.4	Adding an interrupt	61

15.5 Changing the timing model	62
16 Known gaps	63
16.1 CPU and timing	63
16.2 Memory, DMA and the bus	63
16.3 Mappers	64
16.4 PPU	64
16.5 APU	64
16.6 CGB	64
16.7 SGB	65
16.8 Front ends	65
16.9 Stale documentation	65
A Opcode coverage	66
B I/O register map	67
C Public API reference	68
C.1 CPU	68
C.2 Bus	68
C.3 Timer, PPU and APU	68
C.4 Loop and persistence	69
D Constants	70
E Further reading	71
E.1 Reference documentation	71
E.2 Test ROMs	71
E.3 Internal notes	71
F Index	72

1 Introduction

1.1 What gbemu is

gbemu is a from-scratch emulator for the original monochrome Game Boy and, partially, for the Game Boy Color. It is written in C99 with no mandatory third-party dependency: the core library talks to nothing but the C standard library. Optional back ends add SDL3 (desktop), emscripten (browser) or ESP-IDF (microcontroller) support, but each one is a separate, replaceable translation unit.

The design goal is *testability first*. Every hardware subsystem is a small, independently callable state machine that advances by an explicit number of cycles, so that conformance ROMs can be run at full speed with no window and no sound card. The second goal is *transparency*: the source is small enough to read in an afternoon, and this manual explains every register, every timing constant and every deliberate deviation from real hardware.

Scope: The emulator implements the DMG feature set, the four supported memory bank controllers, the four audio channels, and the interrupt system. Game Boy Color support is limited to the CPU speed switch; the CGB-specific palette, video and DMA registers are not implemented and are listed in section 16.6.

1.2 Design goals

- Frontend independence : The core exposes a four-function vtable (`init`, `render`, `poll_events`, `destroy`) through `gb\_frontend`, plus an optional hotkey callback. A frontend never pokes at emulator internals; it receives a frame buffer and a memory bus. This is what makes the headless test build possible.
- Cycle-driven hardware : The CPU does not "run" the hardware; hardware is advanced explicitly. `gb\_cpu\_step` reports how many T-cycles an instruction took and calls `gb\_machine\_tick` at each machine-cycle boundary, so the timer, the PPU, the APU and the OAM DMA engine all observe bus activity at the right slot in time.
- Generated opcode tables : Opcode mnemonics and cycle counts come from a machine-readable table and are regenerated into `src/opcodes.h` by `scripts/generate\_opcodes.py`. Hand-maintained opcode lists drift; generated ones do not.
- Documented from the source : The public header `src/gbemu.h` carries structured comments (`// @func`, `// @param`, `// @desc`, ...) that are turned into browsable API documentation by `makedocs`. This manual covers the parts a header comment cannot: why the code looks the way it does, and what it does not do.
- Portability : Only the core library has to be portable. The timing helpers (`gb\_get\_time\_ns`, `gb\_sleep\_ns`) switch between POSIX and FreeRTOS behind `#ifdef`, and the ESP32 build compiles the core freestanding, with no ESP-IDF headers.

1.3 Feature summary

Area	What is implemented
CPU	Full SM83 instruction set: all 256 base opcodes and all 256 CB-prefixed opcodes, with hardware-accurate cycle counts, the EI one-instruction delay, the HALT bug and interrupt dispatch.
Memory bus	Complete address decoding including echo RAM, the unusable region, I/O register routing, the joypad matrix, OAM DMA and a flat I/O shadow array for registers that are not modelled.
Cartridges	ROM-only, MBC1 (including MBC1M multicarts), MBC2, MBC3 (with an RTC register stub) and MBC5. Battery saves for every mapper that has battery-backed RAM.
Timer	DIV/TIMA/TMA/TAC with a 16-bit internal counter, a bit-selected frequency clock and a falling-edge detector, including the delayed TIMA reload and the DIV-write trigger.
PPU	Background, window and sprite rendering to a 160×144 ARGB frame buffer, scanline mode transitions (OAM, transfer, H-blank, V-blank), L Y/LYC coincidence and the four STAT interrupt sources.
Joypad	P1 at 0xFF00 with the two select lines, active-low button state and joypad interrupt generation.
OAM DMA	0xFF46 transfers copy 160 bytes at one byte per machine cycle after a two-cycle start delay, with restart queueing and OAM blocking during the copy.
Audio	Four-channel synthesis (two squares with duty, a wave channel reading channel 3's sample RAM, and a 15/7-bit LFSR noise channel), NR51/NR50 panning and master volume, naive decimation to the output rate and a single-producer/single-consumer ring buffer.
Serial	Output is intercepted: a write of 0x81 to 0xFF02 prints the pending byte, and writes to cartridge RAM above 0xA004 print printable bytes. This is what makes the Blargg, mooneye and SameSuite suites reportable.
Persistence	Battery saves (<rom>.sav) and full-machine save states (<rom>.state) with version and ROM-hash validation.
Frontends	SDL3, terminal (ANSI truecolour half-blocks), headless, WebAssembly and ESP32.

1.4 Accuracy and known limitations

An emulator that documents only its successes is not much use to anyone trying to decide whether a given test failure is the emulator's fault. The known deviations are therefore listed where they belong, in the chapter of the subsystem they affect:

- **CPU:** STOP consumes its operand byte and performs the CGB speed switch, but it does not stop the CPU, the divider or the

APU.

- **Bus:** unmapped I/O reads return 0x00 rather than 0xFF; VRAM and OAM are not blocked during PPU modes 2 and 3; the OAM DMA source page is not masked or validated; 0xFF46 does not read back the value written to it.
- **Timer:** writing TAC does not generate the falling edge that real hardware produces.
- **PPU:** scanlines are rendered in one step at dot 504 rather than pixel by pixel, mode 3 has a fixed length, LCDC bit 0 is ignored, the sprite priority flag (OAM attribute bit 7) is parsed but not applied, scanline 0 is not drawn after the LCD is switched on, and there is no OAM-scan corruption bug.
- **APU:** the frame sequencer period, the noise channel shift and the wave-channel volume register are all wrong in ways section 10.11 quantifies; the volume envelope is not wired up.
- **CGB:** no palette RAM, no VRAM banking, no GDMA/HDMA, no boot ROM. See section 16.6.

1.5 Repository layout

Path	Contents
src/	The emulator core. <code>gbemu.h</code> is the entire public API; <code>frontend.h</code> declares the vtable. <code>cpu.c</code> , <code>instr.c</code> , <code>opcodes.h</code> (generated), <code>bus.c</code> , <code>timer.c</code> , <code>ppu.c</code> , <code>apu.c</code> , <code>inter.c</code> , <code>loop.c</code> , <code>memops.c</code> and <code>save.c</code> make up the implementation. Nothing in this directory includes SDL or any windowing header.
frontend/	One file per front end: <code>SDL.c</code> , <code>terminal.c</code> , <code>headless.c</code> , <code>wasm.c</code> , <code>esp32.c</code> . Exactly one of these is compiled into a given host program; the build system picks the default.
apps/	Host programs. <code>cli.c</code> is the desktop/CLI entry point, <code>wasm.c</code> is the emscripten entry point with the exported functions the web shell calls.
make/	GNU make fragments: <code>common.mk</code> (compiler and flags), <code>libs.mk</code> (artifacts), <code>utils.mk</code> (housekeeping), <code>wasm.mk</code> , <code>esp32.mk</code> , <code>docs.mk</code> .
scripts/	<code>generate_opcodes.py</code> regenerates <code>src/opcodes.h</code> ; <code>test</code> is the test-suite driver.
tests/	Checked-out test ROMs: <code>gb-test-roms</code> (Bla'rg), <code>mooneye-test-suite</code> and <code>SameSuite</code> .
docs/	This manual (<code>manual.tex</code>), developer notes under <code>docs/dev/</code> , and the generated API reference (<code>tiny.docs.json</code> plus <code>index.html</code>).
web/	<code>shell.html</code> , the emscripten shell template that becomes <code>emu-wasm.html</code> .

1.6 Conventions used in this manual

Identifiers : are set in typewriter font: `gb\_bus\_write`.

- Hexadecimal constants : are written `0xFF40` with an `0x` prefix and four digits for addresses; masks and immediate values may be shorter.
- Time bases : are named explicitly. A *dot* (a system cycle) is a `GB\_CPU\_FREQ` unit at 8.388608 MHz. A *T-cycle* is a CPU clock unit, equal to two dots at normal speed. An *M-cycle* (machine cycle) is four T-cycles, equal to one bus access. Section 4.5 tabulates the conversions.
- Addresses : are given as full I/O addresses where the I/O space is involved (`0xFF05`) and as cartridge-window offsets where the mapper is involved (`$2000--$3FFF`), matching the conventions of the Game Boy hardware documentation.
- DONE/TODO : items in the source `TODO` file are quoted when they explain a missing feature.

1.7 License and authorship

The project is released under the MIT license. The public header declares this too, through the `//@license` documentation marker. Contributions are welcome; the fastest way to make progress is to pick an item from the `TODO` file or a failing conformance test and send a patch.

2 Building gbemu

2.1 Prerequisites

The core needs only a C99 compiler. The rest depends on the target:

Target	Requirements
Core library, CLI	gcc (or any C99 compiler). On macOS with Homebrew this is the system compiler. pkg-config is used to locate SDL3.
SDL3 front end	SDL3 development files, discovered with <code>\pkg-config--cflags--libssdl3</code> . The makefiles additionally point <code>\PKG_CONFIG_PATH</code> at <code>/opt/homebrew/lib/pkgconfig</code> , which is where Homebrew installs it on Apple Silicon.
Terminal / headless	Nothing beyond libc; useful on systems without a display server.
WebAssembly	emcc from the Emscripten SDK. Only the core, the WASM front end and <code>apps/wasm.c</code> are compiled; SDL is not involved.
ESP32	The Espressif cross toolchain, on <code>\PATH</code> after sourcing <code>\\$HOME/esp/esp-idf/export.sh</code> . The core cross-compiles freestanding; <code>frontend/esp32.c</code> additionally needs a full ESP-IDF component environment.
Documentation	pdflatex for this manual, tinydocs-cli for the API reference, rgbds (for SameSuite) and wla-dx (for mooneye) if you want to rebuild the test ROMs.

2.2 Build targets

The top-level `Makefile` includes five fragments and defines `\help` as the default goal. Every target below carries a `\##` comment so that `\makehelp` lists it:

Target	Description
<code>\all</code>	The default build: checks the toolchain, creates <code>build/</code> , then builds the static library, the shared library and the CLI binary. Equivalent to <code>\check_toolsstaticsharedgbemu-cli</code> .
<code>\static</code>	Archives the core objects into <code>libgbemu.a</code> with <code>\arrcs</code> .
<code>\shared</code>	Links the same objects into <code>libgbemu.so</code> with <code>\-shared</code> .
<code>\gbemu-cli</code>	Links <code>apps/cli.c</code> together with the selected front end against <code>libgbemu.a</code> . This is the program you actually run.
<code>\check_tools</code>	Verifies that gcc exists (fatal otherwise) and warns if bear is missing, since that only affects the <code>\compile.commands.json</code> target.
<code>\clean</code>	Removes <code>build/</code> , the binary and both libraries.
<code>\distclean</code>	<code>\clean</code> plus editor backups, core dumps and <code>dist/*.tar.gz</code> .

Target	Description
<code>\dist</code>	Packs <code>src</code> , <code>frontend</code> , <code>apps</code> , the Makefile and <code>README.md</code> into <code>dist/gbemu-cli.tar.gz</code> .
<code>\compile_commands.json</code>	Regenerates the compile database used by <code>clangd</code> and IDEs, via <code>\bear--makeall</code> .
<code>\docs</code>	Runs <code>tinydocs-cli</code> over <code>src/gbemu.h</code> and <code>src/frontend.h</code> and writes <code>docs/tiny.docs.json</code> plus an HTML page.
<code>\manual</code>	Compiles this document: three <code>pdflatex</code> passes around a <code>makeindex</code> run, with the output collected in <code>docs/manual/</code> (see section 2.8).
<code>\manual.clean</code>	Removes <code>docs/manual/</code> .
<code>\install / \uninstall</code>	Copy, respectively remove, the binary in <code>\PREFIX</code> , which defaults to <code>\usr/local/bin</code> . Both need write access, i.e. usually <code>\sudo</code> .
<code>\wasm</code>	Cross-compiles the core plus the WASM front end with <code>emcc</code> into <code>web/emu-wasm.html</code> and its sidecar files.
<code>\wasm.serve</code>	Rebuilds the WASM target and serves <code>web/</code> on <code>\http://localhost:8080</code> with <code>\python3-mhttp.server</code> .
<code>\wasm.clean</code>	Removes the WASM objects and every <code>emu-wasm.*</code> artefact.
<code>\esp32-lib</code>	Cross-compiles the core for the microcontroller into <code>build/esp32/libgbemu-<target>.a</code> .
<code>\esp32-clean</code>	Removes <code>build/esp32</code> .

Variables: The build accepts three meaningful variables. `\type=RELEASE` switches to an optimised, sanitizer-free build. `\TERM=1` and `\HEADLESS=1` select the terminal and headless front ends instead of `SDL3`. `\WASM_ROM=<path>` preloads a ROM into the WASM virtual file system, and `\ESP32_TARGET=<chip>` picks the cross toolchain. `\PREFIX` overrides the install directory.

2.3 Build types

`\make/common.mk` treats `\type=RELEASE` specially:

Debug (default) : `\-DDEBUG-ggdb` plus `\-fsanitize=address,undefined` in both the compile and the link line. This is the build you want while developing: the sanitizers catch out-of-bounds accesses in the frame buffer and in the DMA code that would otherwise show up as a corrupted picture two frames later.

Release : `\makealltype=RELEASE` compiles with `\-O3` and no sanitizers. Everything always compiles with `\-Wall`, `\-Isrc` and `\-fPIC` (the latter so the objects can go into the shared library).

2.4 Choosing a front end at build time

`\make/common.mk` inspects two variables and sets exactly one front-end source file, plus the matching `\-D` macro that `apps/cli.c` tests:

Variable	Define	Front end
(not set)	—	<code>frontend/sdl.c</code> , compiled together with the SDL3 flags reported by <code>\pkg-config</code>
<code>\TERM=1</code>	<code>\-DEMU_TERM</code>	<code>frontend/terminal.c</code> ; no SDL dependency at all, so this is the build to use on a headless box
<code>\HEADLESS=1</code>	<code>\-DEMU_HEADLESS</code>	<code>frontend/headless.c</code> ; initialises nothing, renders nothing, never terminates on its own. The test harness uses this.

The front-end factory function differs per build, which is why `apps/cli.c` declares three of them and links only one:

```
gb_frontend* frontend_sdl_create(gb_apu* apu);
gb_frontend* frontend_headless_create(void);
gb_frontend* frontend_terminal_create(void);
```

Note that the SDL constructor takes a `gb\_apu*` because its audio callback needs the ring buffer; the other two take nothing and produce no sound.

2.5 Build outputs

`\makeall` leaves three artefacts in the repository root:

Artefact	Description
<code>gbemu-cli</code>	The executable. Takes a ROM path as its only argument.
<code>libgbemu.a</code>	Static library containing the core only — no front end, no <code>\main</code> . This is the artefact to link into an application of your own.
<code>libgbemu.so</code>	The same objects as a shared library, for run-time loading or for embedding in another language.

2.6 The WebAssembly target

`\makewasm` compiles every core source *except* `loop.c` (the browser supplies its own frame loop), adds `frontend/wasm.c` and `apps/wasm.c`, and links with `emcc`. The link options matter:

```
-s ALLOW_MEMORY_GROWTH
-s INITIAL_MEMORY=16777216
-s EXPORTED_RUNTIME_METHODS='["FS","HEAPU8"]'
-s EXPORTED_FUNCTIONS=['_main','_wasm_set_key','_wasm_load_rom','_wasm_is_running']
--shell-file web/shell.html
```

Two consequences follow. `FS` is required because the page writes the chosen ROM into the virtual file system as `/rom.gb` before calling `\_wasm\_load\_rom`. `HEAPU8` is exported because the front end wraps a pointer into the WASM linear memory to hand it to JavaScript. `\INITIAL_MEMORY` of 16 MiB leaves room for a large ROM plus its bank-aligned allocation; `\ALLOW_MEMORY_GROWTH` lets a huge cartridge still fit.

Debug builds add `\-sASSERTIONS=2` so that a null pointer or a misaligned access traps instead of corrupting memory silently.

Passing `\WASM_ROM=path/to/rom.gb` adds emcc's `\--preload-file` option with the target `@/rom.gb`, so the page boots straight into a game without a file picker.

2.7 The ESP32 target

`\makeesp32-lib` compiles the core for the microcontroller and archives it:

```
make esp32-lib # xtensa-esp32-elf-gcc
make ESP32_TARGET=esp32s3 esp32-lib
make ESP32_TARGET=esp32c3 esp32-lib # riscv32-esp-elf-gcc
```

Supported `\ESP32_TARGET` values are `\esp32`, `\esp32s2`, `\esp32s3` and `\esp32c3`; anything else is a hard make error. The compile flags are deliberately minimal: `\-O2`, `\ffunction-sections` and `\fdata-sections` so the linker can garbage-collect unused code, plus `\-DESP_PLATFORM`, which is the marker that switches the portable paths inside `loop.c` and `bus.c`:

- `gb\_sleep\_ns` uses `\vTaskDelay` instead of `\nanosleep`, selected by a compile-time probe for `\<freertos/FreeRTOS.h>`.
- The `\EMU_NOSLEEP` environment variable is never read; frame pacing is unconditional.
- The Bla'rg trace file side effect is compiled out.

The core needs no ESP-IDF headers, so the archive links into any IDF project. `frontend/esp32.c` is *not* part of this target: it needs the full component environment, and section 12.5 describes how to use it from your own firmware.

2.8 Building this manual

`\makemanual` compiles `docs/manual.tex` into `docs/manual/`; the target is defined in `make/docs.mk` and shells out to the toolchain below. The document uses only packages from a standard TeX Live installation (`\refart` with `\longtable`, `\booktabs`, `\array`, `\fancyvrb` and `\hyperref`), and it has to run the LaTeX pass more than once so that the index and the page references settle:

```
cd docs
pdflatex -interaction=nonstopmode -output-directory=manual manual.tex
makeindex -o manual/manual.idx manual/manual.idx
pdflatex -interaction=nonstopmode -output-directory=manual manual.tex
pdflatex -interaction=nonstopmode -output-directory=manual manual.tex
```

`\makemanual` does exactly this, including the `\-output-directory`, so the `.aux`, `.log` and `.idx` files never appear next to the source. If you invoke `\pdflatex` by hand without that flag, compile in `docs/` instead and clean up the intermediate files afterwards with `latexmk-c` or by hand.

! The index is only complete if `\makeindex` runs *after* the first LaTeX pass has written `manual.idx` and *before* the last one. Skipping it produces a document that compiles cleanly and prints an empty index.

The layout is the reference-manual style: the text occupies a narrow column on the right, and the wide left margin carries `\marginlabel` annotations, `\seealso` cross references and `\attention` notes. If you extend the document, that is where the asides belong.

3 Running the emulator

3.1 Command line

The CLI takes exactly one argument, the path to a ROM file:

```
$ ./gbemu-cli roms/"Tetris (World) (Rev 1).gb"
```

With no argument it prints a usage message and exits with status 1:

```
Please provide a rom
Usage: ./gbemu-cli <ROM>
```

Paths are used verbatim, including spaces — quote them. The ROM's directory matters, because both the battery save and the save state are written next to the ROM (section 11).

3.2 Environment variables

Variable	Default	Meaning
\EMU_SCALE	\4	Integer window scale for the SDL front end; the window is \160*scale by \144*scale. Values below 1 are clamped to 1. \EMU_SCALE=3 gives a 480×432 window.
\EMU_NOSLEEP	unset	If the variable exists at all — its value is ignored — the emulator does not sleep between frames and runs as fast as it can. This is what the test harness sets, and what you want when measuring emulation speed. It is not read at all in the ESP32 build.

3.3 Controls

The joypad state is a pair of bytes on the bus, `joypad\_dpad` and `joypad\_buttons`, both *active low*: a set bit means released. Only the low nibble of each byte is used.

Button	Field	Bit	SDL3 key
Right	joypad_dpad	0x01	\SDLK_RIGHT
Left	joypad_dpad	0x02	\SDLK_LEFT
Up	joypad_dpad	0x04	\SDLK_UP
Down	joypad_dpad	0x08	\SDLK_DOWN
A	joypad_buttons	0x01	\SDLK_Z
B	joypad_buttons	0x02	\SDLK_X
Select	joypad_buttons	0x04	\SDLK_BACKSPACE
Start	joypad_buttons	0x08	\SDLK_RETURN

Any SDL key press raises the joypad interrupt flag; the ESP32 front end detects the 1-to-0 transitions properly instead, and the terminal front end raises it once per poll unconditionally (a known quirk, section 12.2).

3.4 Hotkeys

The front end never decides what a hotkey *means*. It recognises the input and emits an action through the `on\_hotkey` callback; the application decides that a save action should call `gb\_save\_state`.

Action	SDL3	ESP32
GB_HOTKEY_SA VE_STATE	\SDLK_F5	\START+SELECT held for about half a second
GB_HOTKEY_LO AD_STATE	\SDLK_F9	\START+SELECT held for about two and a half seconds

Terminal, headless and WASM builds install no hotkey callback at all, so the keys do nothing there. The full wiring is shown in section 13.3.

3.5 Battery saves and save states

Both kinds of file are derived from the ROM path by appending an extension:

File	Written	Contains
<rom>.sav	Automatically when <code>gb_loop</code> returns, for cartridges whose header declares a battery	Cartridge RAM, and the RTC registers for MBC3 timer carts
<rom>.state	On \F5	The complete machine state: CPU, bus, timer, PPU, APU

On start-up the emulator loads <rom>.sav if it exists. Save states are loaded only when you press the load hotkey, and are rejected unless they were written by the same emulator version from the same ROM (section 11.3).

3.6 Shutting down

The emulation loop keeps a local `running` flag which only the front end can clear, through the `\running` out-parameter of `poll\_events`. The SDL front end clears it on `\SDL_EVENT_QUIT` or a window-close request; the terminal front end clears it on `\Ctrl-C`, on a bare escape, or on `\q`. Once the loop exits, `\main` writes the battery save, then destroys the front end and returns.

Two consequences are worth knowing. Input is only sampled once per frame, so quitting takes effect within roughly one frame. And because the headless front end never clears the flag, a headless process runs until it is killed — which is exactly what the test harness relies on.

4.1 Layers

```
+-----+
| host program          apps/cli.c      apps/wasm.c           |
| main(), state objects, hotkey policy, exit handling         |
+-----+
|                               gb_frontend* (vtable)            |
+-----+
| front end             frontend/sdl.c, terminal.c,           |
| pixels, audio, input   headless.c, wasm.c, esp32.c          |
+-----+
|                               gb_bus*, gb_apu*                |
+-----+
| core library          src/cpu.c instr.c bus.c timer.c       |
| (libgbemu.a / .so)    src/ppu.c apu.c inter.c loop.c        |
|                        src/memops.c save.c                    |
+-----+
```

4.2 Module map

gbemu Reference Manual

File	Responsibility
memops.c	Two helpers, gb_fetch8 and gb_fetch16, which tick one machine cycle per byte read and assemble little-endian 16-bit values.
save.c	Battery saves and save states.

4.3 The front-end vtable

src/frontend.h declares the whole interface between core and host:

```
typedef enum {
    GB_HOTKEY_SAVE_STATE, // Quick-save the full machine state (F5)
    GB_HOTKEY_LOAD_STATE, // Restore the last quick-save (F9)
} gb_hotkey;

struct gb_frontend {
    void* priv;

    bool (*init)(gb_frontend* fe, int width, int height);
    void (*render)(gb_frontend* fe, const uint32_t* buffer, int width, int height);
    void (*poll_events)(gb_frontend* fe, gb_bus* bus, bool* running);
    void (*destroy)(gb_frontend* fe);

    void* hotkey_ctx;
    void (*on_hotkey)(void* ctx, gb_hotkey key);
};
```

The contract, call by call:

Slot	Called	Contract
init	Once, by \main, with \160 and \144	Returns \true on success. Allocate everything private here; return \false to abort start-up, which makes \main exit with status 1.
render	Once per frame, only when \ppu->frame_ready was set	\buffer points at \160*144 ARGB words in row-major order. The core clears the flag after the call, so the front end must copy what it needs rather than keep the pointer.
poll_events	Once per frame, after rendering	Drain the platform event queue, write the joypad bits into \bus->joypad_dpad / \bus->joypad_buttons (active low), raise \bus->joypad_interrupt when a button was newly pressed, and clear *running to end the session.
destroy	Once, by \main after the loop	Release everything \init allocated. The core does not touch the vtable afterwards.
on_hotkey	From \poll_events, if non-NULL	The front end signals an intent; the application performs the save or load. May be \NULL.

Ordering: Rendering strictly precedes event polling, and both happen after the cycle budget for the frame is spent. Input therefore takes effect on the following frame, at most 16.7 ms later.

4.4 The execution model

The core is a cycle-driven state machine. Nothing in it is event-driven: there are no callbacks from the timer into the CPU, no interrupts-as-preemption, no coroutines. Each component is a plain struct plus a `\step` function that takes a cycle count.

The call graph of one frame of `gb\loop`:

```

gb_loop(cpu, bus, timer, ppu, apu, fe)
|
+- gb_get_time_ns()                                frame start timestamp
|
+- while (frame_cycles < GB_CYCLES_PER_FRAME):
| |
| | +- gb_cpu_step(cpu, bus) -----> T-cycles
| | | +- promote ime_scheduled -> ime
| | | +- if halted: gb_machine_tick(bus,4); return 4
| | | +- gb_machine_tick(bus, 4)                                M-cycle 1
| | | +- opcode = gb_bus_read(bus, pc); pc++                    (skipped on halt bug)
| | | +- gb_instr(cpu, bus, opcode)
| | | | +- 0xCB ? instr_cb() : handler chain
| | | | +- handler calls gb_machine_tick(bus,4) per M-cycle
| | | | +- handlers call gb_fetch8/16 (which tick too)
| | +- gb_handle_interrupts(cpu, bus, ppu, timer) -----> T-cycles
| | | +- latch PPU/timer/joystick requests into IF (0xFF0F)
| | | +- wake the CPU if IF & IE != 0
| | | +- if IME: push PC, jump to 0x40 + 8*bit, cost 20 T-cycles
| | +- frame_cycles += t_cycles * (double_speed ? 1 : 2)
|
+- if (ppu->frame_ready): fe->render(...); frame_ready = false
+- fe->poll_events(fe, bus, &running)
+- gb_get_time_ns(); gb_sleep_ns(remaining of 1/60 s) unless EMU_NOSLEEP

```

Four properties of this design are worth stating explicitly, because they explain most of the code.

Cycles are returned, never accumulated : `gb\cpu\step` returns the number of T-cycles an instruction consumed, taken from the generated table *before* execution starts. Handlers report only whether they handled the opcode. Because the returned total is fixed, conditional branches are charged their taken or untaken cost based on a *pre-execution* evaluation of the flag — which is why `get\condition\met` is called before any handler runs.

Hardware advances at machine-cycle boundaries : `gb\machine\tick` is called once for the opcode fetch and once for every subsequent bus access. It steps the DMA engine, the timer, the PPU and the APU. A load from [HL] costs a machine cycle because the register-index helper calls it explicitly, which is what makes the `\BITn`, (HL) timing come out right.

Interrupts are checked between instructions : `gb\handle_interrupts` runs after every completed instruction, so worst-case interrupt latency is one instruction. Only one interrupt is dispatched per call; the remaining pending bits stay in IF for the next instruction boundary.

A frame is a cycle budget, not a V-blank : The inner loop runs until `\GB_CYCLES_PER_FRAME` dots have been spent, and renders if — and only if — the PPU raised `\frame_ready` during that budget. If a frame overruns, the next render simply shows the newer picture; the flag is cleared on read, so a second V-blank inside one budget is dropped rather than queued.

4.5 Time bases

Unit	Magnitude	Notes
Dot (system cycle)	8.388608 MHz	<code>\GB_CPU_FREQ</code> . The PPU and the APU count dots; the frame budget is expressed in dots, which is why it is speed-mode independent.
T-cycle	4.194304 MHz	Two dots at normal speed, one dot in CGB double-speed mode. The CPU and the timer live in this domain; the timer is <i>not</i> rescaled, so <code>DIV</code> ticks twice as fast in real time in double-speed mode, exactly as on hardware.
M-cycle	4 T-cycles	One bus access. <code>gb_machine_tick</code> is always called with <code>\4</code> .
Frame	$\frac{\text{\GB_CYCLES_PER_FRAME}}{\text{\GB_CPU_FREQ}} = \frac{139\,810}{60} = 2330 \text{ dots}$	The inner-loop budget.
Frame wall clock	$\frac{\text{\GB_FRAME_TIME_NS}}{10^9} = \frac{16\,666\,666}{10^9} = 16.666666 \text{ ns}$	The pacing target.
Scanline	912 dots = 456 T-cycles	See section 9.2.
Interrupt dispatch	20 T-cycles = 5 M-cycles	See section 5.8.

4.6 State objects and ownership

An application owns five objects and keeps them alive for the whole session:

```
gb_cpu  cpu;    // registers, IME, halt state
gb_bus  bus;    // memory map, cartridge, MBC state, joypad, DMA
gb_timer timer; // DIV/TIMA/TMA/TAC and the internal counter
gb_ppu  ppu;    // video registers and the frame buffer
gb_apu  apu;    // sound registers, channel state, ring buffer
```

They must be wired together before the first tick:

```
bus.timer = &timer;
bus.ppu   = &ppu;
bus.apu   = &apu;
```

`! gb\_machine\_tick` guards each of these with an `\if (bus->ppu)` style test. If you forget a pointer, the machine does not crash — it silently stops advancing that component, and you get a black screen with no error message.

Two other ownership rules:

- `gb\_bus\_load\_rom` allocates `\bus->rom` and `\bus->sram` itself and frees any previous allocation, so a reload is safe. Nothing else allocates.

- The PPU frame buffer is the only large allocation in the state objects ($160 \times 144 \times 4$ bytes, about 90 KiB). It is deliberately placed last in the struct so that save states can serialise the register prefix with a single `\offsetof`.

4.7 Symbol prefixing and `\GBEMU_STRIP_PREFIX`

Every public symbol carries a `gb\_` prefix and every macro a `GB\_` prefix: `gb\_bus\_write`, `gb\_ppu\_step`, `GB\_SCREEN\_WIDTH`. The enum constants (`GB\_FLAG\_Z`, `GB\_PPU\_MODE\_VBLANK`) follow the same rule. This keeps the header usable in projects that already have a symbol called `loop` or `version`.

If you are writing a single-file program that links nothing else, you can ask the header to undo the prefixing:

```
cc -DGBEMU_STRIP_PREFIX my_game.c -I src libgbemu.a -o my_game -lSDL3
```

After that, `cpu\_init` expands to `gb\_cpu\_init`, `CPU` to `gb\_cpu`, `SCREEN\_WIDTH` to `GB\_SCREEN\_WIDTH`, and so on. The mechanism is a flat list of object-like `\#define` aliases placed at the very end of `gbemu.h`, after the real declarations, so the ABI never changes.

! This deliberately gives up namespacing. It redefines very common words (`loop`, `version`, `STR`, `loop`) process-wide. Pass it on the command line as shown rather than defining it in a source file, so that it cannot leak into unrelated translation units, and never enable it in a project that also uses another library with those names.

Only the prefixed spellings are aliased for the types that are also declared in `frontend.h`; including either header alone gives you the same short names.

5 The CPU

5.1 Register file

`gb\_cpu` mirrors the hardware with four anonymous unions, one per register pair:

```
typedef struct {
    union { struct { uint8_t f; uint8_t a; }; uint16_t af; };
    union { struct { uint8_t c; uint8_t b; }; uint16_t bc; };
    union { struct { uint8_t e; uint8_t d; }; uint16_t de; };
    union { struct { uint8_t l; uint8_t h; }; uint16_t hl; };
    uint16_t sp;
    uint16_t pc;
    bool ime;           // Interrupt Master Enable
    bool ime_scheduled; // EI has a 1-instruction delay
    bool halted;
    bool halt_bug;      // HALT with IME=0 and an interrupt pending
} gb_cpu;
```

Byte order: In every pair the *low* register occupies the low byte, so `af=0x11B0` means `f==0xB0` and `a==0x11`. The 16-bit value therefore matches the hardware's `AF` while the storage order is reversed; code that needs `A` from `af` uses `\cpu->af>>8`. `POPAF` masks the low nibble of `F`, as required.

The post-boot register set:

Register	Value	Rationale
\AF	\0x11B0	The Game Boy Color boot ROM leaves <code>A=0x11</code> , which selects the enhanced register set. The application overrides it to <code>\0x01</code> for DMG-only carts by checking header byte <code>0x0143</code> .
\BC	\0x0013	Boot ROM value
\DE	\0x00D8	Boot ROM value
\HL	\0x014D	Boot ROM value
\SP	\0xFFFFE	Just below the I/O block, as after boot
\PC	\0x0100	Execution begins at the cartridge entry point; no boot ROM is emulated.
\timer.intern al_counter	\0xABCC	Chosen so that the upper byte, which is what <code>DIV</code> reads, equals the post-boot <code>\0xAB</code> .
\bus.io[0x0F]	\0x01	<code>IF</code> : a V-blank request left over from the last scanline of the boot ROM. Set by the application, not by the core.
\bus.ie	\0x00	<code>IE</code> : no interrupt source is enabled out of boot.
\joypad.buttons, \joypad_data	\0x0F	All buttons released, given the active-low encoding.

The first six rows are set by `gb\_cpu\_init` and `gb\_timer\_init`; the rest are the host program's responsibility, and both `apps/cli.c` and `aps/wasm.c` set them identically.

5.2 Flags

Flag	Bit	Set when
\GB_FLAG_Z	7	The result of the operation is zero
\GB_FLAG_N	6	The last operation was a subtraction
\GB_FLAG_H	5	Half-carry or half-borrow out of bit 3
\GB_FLAG_C	4	Carry or borrow out of bit 7

Bits 0–3 of F do not exist and always read as zero; POPAF enforces this. `gb\_flag\_set` and `gb\_flag\_get` are the only sanctioned way to touch them, and all of `instr.c` goes through them — 137 call sites, which makes the flag semantics auditable in one place.

5.3 Register-index helpers

Most opcodes encode their operand in three bits. `gb\_get\_reg\_by\_index` and `gb\_set\_reg\_by\_index` turn that index into a register:

Index	Register	Note
\0	\B	
\1	\C	
\2	\D	
\3	\E	
\4	\H	
\5	\L	
\6	\(HL)	The only index with a side effect: a read or write of the bus at address H L, costing one machine cycle.
\7	\A	

Because index 6 pays a machine cycle, `\BITn,(HL)` costs 12 T-cycles (three machine cycles: fetch, prefix byte, memory read) and `\RES/\SETn,(HL)` cost 16 (four), while the register forms cost one and two respectively. The timing falls out of the data path rather than being special-cased.

5.4 Instruction dispatch

`gb\_instr` is the dispatcher. It looks the cycle cost up first, then walks a table of eleven handlers until one of them claims the opcode:

```
static instrfn instr_handlers[] = {
    instr_8bit_arithm, instr_16bit_arithm, instr_bit_flag,
    instr_bit_shift,   instr_bitwise,     instr_carry,
    instr_interrupts,  instr_jumps,        instr_load,
    instr_misc,        instr_stack
};

int gb_instr(gb_cpu* cpu, gb_bus* bus, uint8_t opcode) {
    if (opcode == OP_PREFIX) return instr_cb(cpu, bus);
    int cycles = get_opcode_cycles(opcode, get_condition_met(cpu, opcode));
    for (size_t i = 0; i < N; ++i)
        if (instr_handlers[i](cpu, bus, opcode)) return cycles;
    return 0; // no handler claimed it
}
```

Each handler has the same shape: one `\switch` over the opcode, with fall-through `\case` labels grouping the register variants of one mnemonic, and a `\default:returnfalse;` to decline. Inside a body the register index is recovered arithmetically with `\opcode&0x07` (source operand) and `(\opcode>>3)&0x07` (destination, bit position, or RST vector class).

Handler	Opcodes	Contents
<code>instr_8bit_arithm</code>	61	<code>\ADDADCSUBSBCANDXORORCP</code> against every register and immediate, plus <code>\INC / \DEC</code> in all their forms
<code>instr_load</code>	92	Every <code>\LD</code> variant: register, immediate, <code>\[HL]</code> , <code>\[HL+]</code> , <code>\[HL-]</code> , <code>\LDH</code> , <code>\[a16]</code> , <code>\(C)</code> , and the high-memory forms
<code>instr_16bit_arithm</code>	13	<code>\ADDHL,r16</code> , <code>\ADDSP,e8</code> , <code>\INCr16</code> , <code>\DECr16</code>
<code>instr_bitwise</code>	28	<code>\ANDORXORCPL</code> against every register and immediate, including <code>\CP</code> against <code>\(HL)</code>
<code>instr_bit_flag</code>	0	An empty placeholder: a <code>\switch</code> with only a <code>\default</code> that declines. It sits in the dispatch table, so every instruction pays one extra indirect call for it.
<code>instr_bit_shift</code>	4	Only the accumulator forms <code>\RLCA</code> , <code>\RRCA</code> , <code>\RLA</code> , <code>\RRA</code> ; the register forms are reached through <code>\CB</code>
<code>instr_jumps</code>	30	<code>\JP</code> , <code>\JR</code> , <code>\CALL</code> , <code>\RET</code> , <code>\RETI</code> , <code>\RST</code> in all their conditional and unconditional forms
<code>instr_carry</code>	2	<code>\CCF</code> , <code>\SCF</code>
<code>instr_interrupts</code>	3	<code>\DI</code> , <code>\EI</code> , <code>\HALT</code>
<code>instr_stack</code>	8	<code>\PUSHBCDEHLAF</code> and <code>\POPBCDEHLAF</code> ; <code>\LD[n16],SP</code> , <code>\ADDSP,e8</code> and <code>\LDHL,SP+e8</code> live in <code>instr_load</code> and <code>instr_16bit_arithm</code> instead
<code>instr_misc</code>	3	<code>\DAA</code> , <code>\NOP</code> , <code>\STOP</code>
Base opcodes	244	256 minus the <code>\CB</code> prefix byte and the eleven illegal opcodes. The remaining twelve reach the fatal path in <code>gb_cpu_step</code> .

If no handler claims an opcode, `gb\_instr` returns zero and `gb\_cpu\_step` treats that as fatal:

```
fprintf(stderr, "Unhandled opcode: 0x%02X at PC: 0x%04X\n", opcode, pc - 1);
exit(1);
```

That path is reachable only through the eleven illegal opcodes (`\0xD3`,

`\0xDB, \0xDD, \0xE3, \0xE4, \0xEB, \0xEC, \0xED, \0xF4, \0xFC, \0xFD`), which the opcode table names and prices but no handler implements.

5.5 Generated opcode tables

`src/opcodes.h` is machine-generated by `scripts/generate\_opcodes.py` from a JSON opcode table and must never be edited by hand. It contains:

- An anonymous enumeration of all 256 base opcodes, named `\OP_<mnemonic>_<operands>`.
- An anonymous enumeration of all 256 CB-prefixed opcodes, with the prefix byte folded into the value (`\OP_CB_RLC_B=0xCB00`).
- `\get_opcode_cycles(opcode, condition_met)` and `\get_cb_opcode_cycles(cb_opcode)`, both `\staticinline` switches returning the T-cycle cost.

Naming rules, worth knowing when reading `instr.c`:

Rule	Example
Mnemonic in upper case, operands appended	<code>\OP_ADD_A_B</code> for <code>\ADDA, B</code>
<code>\\$</code> becomes <code>\S</code>	<code>\OP_RST_S38</code> for <code>\RST\$38</code>
Non-immediate operands gain <code>_IND</code>	<code>\OP_LD_HL_IND_B</code> for <code>\LD(HL), B</code>
Post-increment operands gain <code>_INC</code>	<code>\OP_LD_HL_INC_A</code> for <code>\LD(HL+), A</code>
Post-decrement operands gain <code>_DEC</code>	<code>\OP_LD_A_HL_DEC</code> for <code>\LDA, (HL-)</code>
Size hints stay verbatim	<code>\OP_JP_a16</code> , <code>\OP_ADD_SP_e8</code> , <code>\OP_STOP_n8</code>

The cycle functions encode the taken/untaken distinction in the table itself. A conditional branch emits `\returncondition_met?12:8;`, and the caller must therefore evaluate the condition *before* running the handler — flags are clobbered by the time the handler returns.

The generated `\OP_*` constants exist mainly for readability; `instr.c` matches on raw opcode numbers and recovers operand indices with shifts and masks, which is both faster and easier to verify against the reference tables.

5.6 \CB-prefixed instructions

The 256 CB opcodes are *not* dispatched through a table. `instr\_cb` decodes the second byte as bit fields:

Field	Bits	Meaning
<code>reg_idx</code>	<code>\0x07</code>	As in section 5.3
<code>bit_pos</code>	<code>\(b>>3)&0x07</code>	Bit number for <code>\BIT</code> , <code>\RES</code> , <code>\SET</code> , and the shift amount for the rotate group

Field	Bits	Meaning
group	\(b>>6)&0x03	Instruction family

Group	Instructions	Notes
\0x00	\RLC, \RRC, \RL, \RR, \SLA, \SRA, \SWAP, \SRL	Chosen by \bit_pos; always writes back
\0x01	\BITn,r	Writes nothing; sets Z from the bit, clears N, sets H; returns immediately
\0x02	\RESn,r	Clears one bit, writes back
\0x03	\SETn,r	Sets one bit, writes back

The operand is fetched *unconditionally*, before the group is dispatched, which is precisely what makes the \[HL] forms cost more: 12 T-cycles for \BITn, (HL) and 16 for \RES/\SETn, (HL), against 8 for the register forms.

5.7 Instruction timing

All instruction costs are multiples of four T-cycles:

T-cycles	Representative instructions
4	\NOP, \SCF, \CCF, \DAA, \CPL, \HALT, \STOP, \RLCA, all single-byte arithmetic on registers (for example \I NCB, \ADDA,B), and every \JRd8
8	Loads and arithmetic with an immediate (\LDA,d8, \ADDA,d8, \CPd8), \RET, \RETcc untaken, \LD[HLI], A, \LDA,[HLD], \LD(C),A, \LDSP,HL, and every \CB operation on a register
12	\JRcc, \POP, \LDH, \LDH(a8),A, \LDHL,SP+e8, \JPcc untaken, \CALLcc untaken, \BITn,(HL)
16	\JPa16, \JPcc taken, \CALLcc taken, \RETcc taken, \PUSH, \ADDSP,e8, \LDr8,[HL], \LD[HL],r8, \RES /\SETn,(HL)
20	\LD[n16],SP
24	\CALLa16

Two prices per conditional: The table entry for a conditional instruction is written `\returncondition_met?taken:untaken;`. Of the 240 non-\CB opcodes, 145 cost 4, 60 cost 8, 18 cost 16, 15 cost 12, and the multi-byte instructions \CALLa16 and \LD[n16],SP cost 24 and 20. Of the 256 \CB opcodes, 224 cost 8, 24 cost 16 and 8 cost 12.

Invariant: For every opcode, the number of `\gb_machine_tick(bus,4)` calls made along the execution path (opcode fetch plus each operand or memory access) equals the table's T-cycle count divided by four. Nothing checks this at run time, so an added access in a handler must be paid for by removing one elsewhere.

5.8 Interrupts

Hardware raises requests as booleans; `gb\_handle\_interrupts` latches them into `IF`, decides whether to wake or to dispatch, and returns the T-cycles consumed.

IF	Source	Raised by	Vector	Priority
\0	V-blank	<code>ppu->vblank_interrupt</code>	<code>\0x0040</code>	1 (highest)
\1	LCD STAT	<code>ppu->stat_interrupt</code>	<code>\0x0048</code>	2
\2	Timer	<code>timer->interrupt_requested</code>	<code>\0x0050</code>	3
\3	Serial	<i>nothing</i>	<code>\0x0058</code>	4
\4	Joypad	<code>bus->joypad_interrupt</code>	<code>\0x0060</code>	5

The serial bit is never set by the hardware model: serial output is handled by interception (section 6.6), not by a shift register. Priority is the lowest set bit, and exactly one interrupt is dispatched per call.

The register semantics:

- `IF` at `0xFF0F` reads back as `\io[0x0F] | 0xE0`: bits 5–7 read as one. Writes are masked with `\0x1F`.
- `IE` at `0xFFFF` is plain storage in all eight bits, read and written unmasked.
- The pending mask is `\IF&IE&0x1F`. The trailing `\0x1F` is essential: without it the unused `IF` bits would appear to be permanently pending.
- A pending interrupt clears `\halted` whether or not `IME` is set.

When `IME` is set and something is pending, dispatch clears `\IF` and `IME` immediately and then consumes five machine cycles (20 T-cycles) in this order:

1. two internal machine cycles (no bus access)
2. decrement `\SP` by two, one machine cycle, after the two internal cycles
3. write the *high* byte of `PC` to `\SP+1`
4. write the *low* byte of `PC` to `\SP`
5. set `\PC` to `\0x0040+8*bit` — one final internal machine cycle

The two stack writes go through `gb\_bus\_write`, so a game with its stack page in cartridge RAM sees the real bus behaviour, including mapper side effects.

! The interrupt-enable tests are the strictest part of the timing model, because they observe `\IE` and the joypad register mid-frame. The matching conformance tests are run by `scripts/test`; the current failure list is in the `TODO` file.

5.9 \EI, \HALT, the HALT bug and \STOP

Two booleans model the enable sequence:

`\ime_scheduled` : is set by `\EI` and cleared by `\DI`.

`\ime` : is set from `\ime_scheduled` at the *top* of `gb\_cpu\_step`, before the halt check.

This gives `\EI` its one-instruction delay: the instruction following `EI` executes with interrupts still masked, and the first instruction after *that* can be interrupted. `\DI` clears both flags, so `\EI` immediately followed by `\DI` never enables interrupts. `\RETI` sets `IME` immediately, as on hardware, and does not touch `\ime_scheduled`.

`HALT` has three distinct outcomes:

Condition	Behaviour
Nothing pending	The CPU sleeps; each <code>gb_cpu_step</code> burns one machine cycle and returns 4 without touching <code>PC</code> .
Pending and <code>\IME=1</code>	The CPU does not sleep. <code>PC</code> stays at <code>\HALT+1</code> and the pending interrupt is dispatched normally after the instruction.
Pending and <code>\IME=0</code>	<i>The HALT bug.</i> The CPU does not sleep and <code>halt_bug</code> is set.

The `HALT` bug is modelled faithfully: the next opcode fetch reads its byte without incrementing `PC`, so that byte is read twice and every subsequent operand read is shifted back one position. The flag is one-shot — it is cleared by the fetch that consumes it — which is exactly the hardware behaviour for the one-byte `\HALT`.

`\STOP` (`\0x10`) consumes its operand byte, then flips `bus->double\_speed` if `KEY1` bit 0 was set, which is the only way the CGB changes CPU speed. It does *not* stop the CPU, halt the divider or mute the APU; those remain open items in the `TODO` list.

Open item: The interaction between the `\EI` delay and `\HALT` is the one interrupt test that is known to fail (SameSuite `interrupt/ei\_delay\_halt`), and is tracked in the `TODO` file.

6 The memory bus

6.1 Address decoding

`gb\_bus\_read` and `gb\_bus\_write` decode the full 16-bit space. The read path, in order:

Range	Read behaviour
0x0000--0x7FFF	Cartridge ROM, through the mapper's bank resolution (<code>bus_read_rom</code>)
0x8000--0x9FFF	<code>\bus->vram[addr-0x8000]</code>
0xA000--0xBFFF	Cartridge RAM, through the mapper (<code>bus_read_sram</code>)
0xC000--0xDFFF	<code>\bus->wram[addr-0xC000]</code>
0xE000--0xFDFD	Echo RAM: <code>\bus->wram[addr-0xE000]</code> , the same 8 KiB seen at 0xC000
0xFE00--0xFEAF	Sprite attribute table, or <code>\0xFF</code> while an OAM DMA transfer is copying
0xFF00--0xFF7F	I/O dispatch (section 6.2)
0xFF80--0xFFFE	<code>\bus->hram[addr-0xFF80]</code> , 127 bytes
0xFFFF	<code>\bus->ie</code>
fallthrough	<code>\0xFF</code> — this is exactly the unusable region 0xFEAF--0xFEFF, the only gap in the 16-bit space

The write path mirrors it, with two differences: writes to the ROM window go to the mapper and never store anything, and writes to the unusable region are dropped explicitly instead of falling through to a read default.

Not modelled: VRAM and OAM are *not* blocked during PPU modes 2 and 3, and no access restrictions are implemented for any other region either. Real games do rely on the restrictions in a handful of places; see section 9.6 for what that costs.

6.2 I/O dispatch

The I/O window is not a flat array. Reads and writes test specific addresses and ranges first, then fall back to a 128-byte shadow array `bus->io[0x80]`:

Address	Handler
0xFF00	Joypad P1
0xFF04--0xFF07	<code>gb_timer_read</code> / <code>gb_timer_write</code>
0xFF10--0xFF3F	<code>gb_apu_read</code> / <code>gb_apu_write</code> (including the wave RAM at 0xFF30-0xFF3F)
0xFF40--0xFF4B	<code>gb_ppu_read</code> / <code>gb_ppu_write</code> , except that a write to 0xFF46 starts an OAM DMA and returns immediately
0xFF4D	<code>\KEY1</code> : bit 7 is synthesised from <code>\bus->double_speed</code> , bit 0 is the speed-switch request
0xFF0F	<code>\IF</code> : reads OR <code>\0xE0</code> , writes mask <code>\0x1F</code>
otherwise	<code>\bus->io[addr-0xFF00]</code>

Gotcha: Every one of the three component pointers is tested before use. A bus whose `\timer`, `\ppu` or `\apu` pointer is `\NULL` falls through to the shadow array, so those registers appear to work — they just do nothing. The same shadow array is what unmapped registers such as `0xFF03`, `0xFF27` and the unimplemented CGB registers land in; they read back whatever was last written, initially `\0x00` rather than the `\0xFF` real hardware returns.

6.3 Loading a ROM

`gb\_bus\_load\_rom` opens the file, sizes it, allocates and clears the ROM buffer, and inspects three header bytes:

```
bus->rom_banks = next power of two >= ceil(file_size / 0x4000), min 2;
bus->rom_size  = rom_banks * 0x4000;          // zero-padded
bus->rom       = calloc(1, bus->rom_size);
fread(bus->rom, 1, file_size, f);
```

Deriving the bank count from the *file size* rather than from header byte `0x0148` is a deliberate simplification: because the count is a power of two, the mapper’s usual masking `\bank & (rom_banks-1)` is always in range, whatever the header claims. It also means a header that lies about its size is ignored.

Offset	Meaning	Use
0x0143	CGB flag	<code>\0x80</code> (CGB enhanced) or <code>\0xC0</code> (CGB only) set <code>\apu->cgb</code> ; the application also uses it to decide whether to force <code>\A=0x01</code> .
0x0147	Cartridge type	Selects the mapper (section 7)
0x0149	RAM size	Sizes the cartridge RAM allocation (section 7.5)
0x0104	Nintendo logo	Read <i>only</i> for the MBC1M multcart heuristic: a 1 MiB MBC1 cart whose four 256 KiB pages contain the logo in at least three of them is a multcart.

Header byte `0x0148` (ROM size), the checksums and the SGB flag are not read at all.

6.4 Sanitising the interrupt vectors

Conformance ROMs deliberately leave the five interrupt vectors empty or pointing at unusable memory, then install their own handlers at run time. The loader rewrites any vector that is still empty when the ROM is loaded:

- a vector byte of `\0x00` becomes `\0xC9` (`\RET`);
- a vector starting with `\0xC3` (`\JPa16`) whose target lies in `0x8000--0xBFFF` or `0xFE00--0xFF7F` becomes `\0xC9`.

Only `\JPa16` is recognised, and the WRAM and HRAM targets are left alone on purpose — the Bla’rg suites copy their runtime into `0xC000` upwards and jump there. Because the hash used for save states is computed over the buffer *after* this rewrite, states stay consistent with each other.

6.5 The joypad register

P1 at 0xFF00 is the classic two-row matrix. Writes store only the two select bits:

Bit	Meaning
6, 7	Read as one; ignored on write
5 (\P15)	Active low. Clear selects the button row
4 (\P14)	Active low. Clear selects the D-pad row
3–0	The selected row, active low: a set bit is a released button

Reads start from \0xC0|select and then *and* the selected rows into the low nibble, so selecting both rows yields \0x0F and selecting neither yields the intersection of both rows.

A write also raises the joypad interrupt when the newly selected row contains a pressed button, which approximates the hardware’s “selected line goes low” edge.

! The comment in `gbemu.h` describing bits 4–7 of the joypad bytes is stale. Only bits 0–3 of `joypad\_dpad` and `joypad\_buttons` are ever used; the upper nibble is not part of the interface. Section 3.3 has the correct table.

6.6 Serial output

There is no serial shift register. Two intercept paths turn a game’s output into text, and both are what makes automated testing possible.

Register path — 0xFF02. : Writing the exact byte \0x81 (internal clock plus start) prints the byte previously written to SB at 0xFF01 with `\putchar` and `\fflush`, and then stores \0x01 in SC so that a ROM polling bit 7 of \SC terminates. This is the protocol used by mooneye and SameSuite: the ROM prints a pass signature of \0305080D1522 or six \0x42 bytes.

RAM path — 0xA004 and above. : Writes to cartridge RAM at or above 0xA004 print every printable byte (\0x20–\0x7E, newline, carriage return, tab) to standard output, and append *every* byte — including NUL terminators — to `/tmp/blargg\_raw.bin`. This is Bla’rg’s `shell.s` text-output protocol.

Raw trace: `/tmp/blargg\_raw.bin` is opened once per process and truncated at first use. It is the way to recover a diagnostic that contains non-printable bytes, and it is compiled out of the ESP32 build.

The RAM path is only reached by carts whose mapper does not return early — MBC1, MBC5 and ROM-only. MBC2 and MBC3 returns skip it, so text output from those mappers is lost.

6.7 OAM DMA

Writing a source page to 0xFF46 starts a transfer of 160 bytes into the sprite attribute table. The timing is explicit:

When	What happens
Write	<code>\dma_active=true</code> , <code>\dma_start_delay=2</code> , <code>\dma_offset=0</code> , source page latched
+1 M-cycle	Delay decrements; OAM is still accessible
+2 M-cycle	Delay reaches zero, the first byte copies — OAM is now blocked
+3 to +161 M-cycles	One byte per M-cycle, 159 more bytes
+162 M-cycle	<code>\dma_offset==0xA0</code> ; the transfer ends and OAM reopens

While a transfer is copying, reads from `0xFE00--0xFE9F` return `\xFF` and writes are dropped. A write to `0xFF46` issued while bytes are already moving is queued: the current transfer continues and a restart begins two machine cycles later. A write during the initial two-cycle delay, or while a restart is already pending, restarts immediately.

The source address is reconstructed per byte as `\(page<<8)|(\offset & 0xFF)` and read through the ordinary bus. It is neither masked nor validated, so a source page of `\xFE` copies `0xFF` bytes — because OAM reads are themselves blocked during the copy — and pages outside the `\0XX00--0XX9F` window are permitted. Section 16.2 lists this among the known gaps.

6.8 `\IF`, `\IE` and the interrupt flags

Covered in section 5.8: `IF` lives in `\bus->io[0x0F]`, `\IE` in `\bus->ie`, and hardware request flags live in the components that raise them. All three are included in save states; the transient `\joypad_interrupt` flag is not.

6.9 The CGB speed switch

`KEY1` at `0xFF4D` is the only CGB register the emulator implements:

- Read: `\(double_speed?0x80:0x00)|(\io[0x4D]&0x01)` — bit 7 reports the *current* mode, bit 0 the prepared one.
- Write: only bit 0 is stored.
- `\STOP` performs the actual switch, flipping `\bus->double_speed` and clearing the request bit.

Once `double\_speed` is set, `gb\_machine\_tick` divides the cycle count handed to the PPU and the APU by two, and `gb\_loop` counts dots instead of T-cycles. The frame budget is therefore identical in both modes, as on hardware. The rest of the CGB — palettes, VRAM banking, `\GDM A/\HDMA`, the boot ROM — is listed in section 16.6.

7 Memory bank controllers

`src/bus.c` implements four mapper families, selected purely by the cartridge type byte at header offset `\0x0147`. There is no fall-back mapper: an unknown type is treated as ROM-only, and the type is stored as zero so that the persistence layer can tell the difference.

Types	Mapper	Identification predicate
<code>\0x01--0x03</code>	MBC1	<code>\type>=0x01&&\type<=0x03</code>
<code>\0x05--0x06</code>	MBC2	<code>\type>=0x05&&\type<=0x06</code>
<code>\0x0F--0x13</code>	MBC3	<code>\type>=0x0F&&\type<=0x13</code>
<code>\0x19--0x1E</code>	MBC5	<code>\type>=0x19&&\type<=0x1E</code>
otherwise	none	<code>\mbc_type</code> is stored as 0

Ranges rather than equality tests are used deliberately, so that the rumble and battery variants (`\0x1B`–`\0x1E`) are covered without a separate case each. Every predicate is inlined into the read and write paths, which keeps `bus\_read\_rom` and `bus\_read\_sram` free of dispatch tables.

Addressing: All four mappers see the same 32 KiB window. A ROM read is always `rom[bank*0x4000+(addr&\0x3FFF)]`, and every mapper's resolution of `\bank` is finally clamped with `\bank&(\rom_banks-1)`. Because `\rom_banks` is always a power of two chosen by `gb\_bus\_load\_rom`, that clamp can never out-of-bounds, whatever the cartridge claims in its header.

7.1 MBC1

MBC1 has two mode-dependent behaviours and one special case, all confined to `mbc1\_bank\_for\_addr`.

Window	ROM mode	RAM mode
<code>\0x0000--0x3FFF</code>	always bank 0	bank $\ll 5$ from the 2-bit register
<code>\0x4000--0x7FFF</code>	<code>\rom_bank (\ram_bank << 5)</code>	<i>identical</i> to ROM mode
Cartridge RAM	bank 0	the 2-bit register

Three hardware details that the code reproduces:

Bank-0 substitution : `\0x00`, `\0x20`, `\0x40` and `\0x60` are replaced by `\0x01`, `\0x21`, `\0x41` and `\0x61` in the switchable window, so bank 0 is only reachable through the fixed region.

The mode bit only affects the fixed region : The upper two ROM bank bits are OR-ed in in *both* modes; the mode select at `\0x6000` changes only what `\0x0000--0x3FFF` shows and which cartridge RAM bank is addressed. This matches real MBC1B1 hardware, which is not what the classic Game Boy manuals describe.

MBC1M multicarts : A 1 MiB MBC1 cart may actually be a multicart with four 256 KiB games. The header cannot tell you, so `gb\_bus\_load\_rom` computes CRC-32 over the 48-byte Nintendo logo at offset `\0x0104` in each of the four pages and sets `\mbc1_multicart` when at least three match the reference value `\0x46195417` — the same heuristic mooneye-gb uses. In that mode the ROM bank register contributes only its low four bits and the 2-bit register selects the game, with bank 0 valid.

The four registers occupy the standard windows:

Window	Field	Effect
0x0000--0x1FFF	RAM enable	Set when the low nibble of the written byte is <code>\0x0A</code>
0x2000--0x3FFF	ROM bank	Low five bits (<code>\mbc1_rom_bank</code>)
0x4000--0x5FFF	2-bit register	Upper ROM bank bits <i>or</i> the RAM bank, depending on the mode
0x6000--0x7FFF	Mode	<code>\0</code> = ROM banking, <code>\1</code> = RAM banking

Cartridge RAM reads return `\0xFF` while RAM is disabled; the read path checks `\mbc1_ram_enable` before anything else.

7.2 MBC2

MBC2 is the odd one out: it has 512 half-bytes of RAM on the cartridge itself, addressable through the same `\0xA000--0xBFFF` window with the low nine bits of the address, and only four bits per byte are meaningful.

- Writes to `\0xA000--0xBFFF` store `\value&0x0F` and return immediately — which means the serial-output interception of section 6.6 never runs for MBC2 games.
- Reads return `\0xF0 | (ram[addr&0x1FF] & 0x0F)` when RAM is enabled and `\0xFF` when it is not. The upper nibble reads as `\0xF` because the register only has four RAM bits; this is what the mooneye `bits\_unused` test checks.
- The mapper registers live in the *fixed* window and are selected by address bit 8: a write with bit 8 clear is the RAM enable (`\0x0A` in the low nibble), a write with bit 8 set is the 4-bit ROM bank.
- Writes to `\0x4000--0x7FFF` are ignored outright, which the moon-eye test suite verifies on MBC2A hardware.
- Bank 0 is substituted by bank 1 in the switchable window, and the result is masked to the cartridge size.

! MBC2 is the one mapper whose 512-byte save file is a different size from the 8 KiB-or-more allocation the loader makes for every other cart (section 7.5), and `gb\_battery\_save` special-cases it accordingly.

7.3 MBC3

MBC3 adds a real-time clock and a register-select trick: the same 2-bit window that selects a RAM bank can instead select one of five RTC registers.

Window	Field	Effect
0x0000--0x1FFF	RAM/RTC enable	Low nibble <code>\0x0A</code> enables both; anything else disables both
0x2000--0x3FFF	ROM bank	Seven bits; a written 0 becomes 1 immediately, so bank 0 is unreachable here
0x4000--0x5FFF	RAM bank / RTC select	<code>\0-\03</code> select a RAM bank, <code>\0x08-\0x0C</code> select the RTC registers
0x6000--0x7FFF	Latch	Writing <code>\0x00</code> then <code>\0x01</code> latches the clock

The RTC registers are kept in `\bus->mbc3_seconds` (0–59), `\minutes` (0–59), `\hours` (0–23), a 9-bit `\day_counter`, a `\day_carry` flag and a `\timer_halt` flag. Writes are range-checked by modulo (`\value%60`, `\value%24`), which is what the RTC register behaviour amounts to for out-of-range values.

Reads and writes to RTC registers are reached through the normal RAM window after selecting them, and they set `\sram_dirty` so that they end up in the battery save (section 11.1).

! The latch sequence only advances a state machine; it does not copy the host clock into the registers. There is a `\TODO` marker for that. A game that latches before reading will read the counters it found at load time, which is stable but not wall-clock accurate.

7.4 MBC5

MBC5 is the simplest of the four — nine ROM bank bits, four RAM bank bits, no mode bit, no bank-0 substitution — and it is the only mapper that can address a 2 MiB cartridge.

Window	Field	Effect
0x0000--0x1FFF	RAM enable	Low nibble <code>\0x0A</code>
0x2000--0x2FFF	ROM bank low	All eight bits, no masking
0x3000--0x3FFF	ROM bank high	Bit 0 only, giving bank numbers up to 511
0x4000--0x5FFF	RAM bank	Low four bits
0x6000--0x7FFF	Rumble	Stored nowhere; ignored

! Because MBC5 does not substitute bank 1 for a bank 0 write, the loader starts an MBC5 cartridge with bank 1 already selected (`\mbc5_rom_bank.1=1`). Without this, the mooneye harness — which copies its memory and friends out of bank 1 into WRAM before switching banks — would fault on its first library call.

7.5 Cartridge RAM sizing

The allocation comes from header byte `\0x0149`, with a floor:

Header	Allocated	Remark
<code>\0x00</code>	8 KiB	No RAM declared, but still allocated
<code>\0x01</code>	8 KiB	2 KiB requested, raised to the floor
<code>\0x02</code>	8 KiB	The only size real carts use here
<code>\0x03</code>	32 KiB	
<code>\0x04</code>	128 KiB	
other	8 KiB	Treated as 8 KiB

! The 8 KiB floor exists for the test suites: Bla'rg's `shell.s` buffers its serial output in SRAM at `\0xA004` and above, so a cartridge that declares no RAM still needs a writable window there or the diagnostic never reaches standard output. The trade-off is that `\sram_banks` is 1 for most carts, so the bank mask is a no-op.

→ `\rom_banks` is computed the same way — the next power of two at or above the number of 16 KiB banks the file actually occupies, minimum 2 (section 6.3).

8 The timer and divider

The timer is the one component that does *not* get its cycle count scaled. `gb\_machine\_tick` hands the raw T-cycle count to `gb\_timer\_step` and the scaled dot count to the PPU and the APU, because `\DIV` is derived from the CPU clock, not from the dot clock. The consequence is that in CGB double-speed mode `\DIV` increments twice as fast in wall-clock terms, exactly as on hardware.

8.1 The divider

`gb\_timer.internal\_counter` is a 16-bit free-running counter incremented once per T-cycle. `DIV` is simply its upper byte, and `gb\_timer\_init` seeds it with `\0xABCC` so that the first `DIV` read returns the post-boot `\0xAB`.

The timer input is a *bit* of that counter selected by `\TAC`, and `TIMA` increments on the falling edge of that bit — the standard hardware model, implemented by sampling the bit before and after the increment:

```
bool bit_before = get_timer_bit(timer); // TAC enable + selected bit
timer->internal_counter++;
bool bit_after = get_timer_bit(timer);
if (bit_before && !bit_after) {
    if (tima == 0xFF) { tima = 0; reload_pending = true; request_interrupt(); }
    else
        tima++;
}
```

Four rates: `\TAC` bits 1–0 select the counter bit, and therefore the `TIMA` increment period in T-cycles:

\TAC bits	Counter bit	Period	Frequency
\00	9	1024 T-cycles	4096 Hz
\01	3	16 T-cycles	262 144 Hz
\10	5	64 T-cycles	65 536 Hz
\11	7	256 T-cycles	16 384 Hz

Bit 2 of `\TAC` is the enable. `gb\_get\_timer\_bit` returns `\false` immediately when it is clear, which means writing a zero to `TAC` can itself produce a falling edge — and therefore increment `TIMA`. That is real hardware behaviour, and the code lets it happen rather than special-casing the register write.

8.2 The `\TIMA` overflow delay

An overflow does not reload `\TIMA` immediately. `tima` is set to `\0x00` and `\tima_reload_pending` is raised; the reload happens on the *next* call to `gb\_timer\_step`, before the counter is incremented:

```
if (timer->tima_reload_pending) {
    timer->tima = timer->tma;
    timer->tima_reload_pending = false;
}
```

That one-cycle delay is what makes the mooneye `timer/tima\_reload` test pass, and it gives a game exactly one cycle in which `TIMA` reads as zero. A write to `TIMA` during that window cancels the pending reload, again matching hardware.

The interrupt request is raised at the moment of the overflow, not at the reload, so the flag can already be in `\IF` one T-cycle before `TIMA` reads `\TMA`.

8.3 Register behaviour

Register	Behaviour
<code>DIV (\0xFF04)</code>	Read: upper byte of the internal counter. Write: <i>any</i> value clears the whole counter, and — if the timer was enabled and the selected bit was 1 — that reset counts as a falling edge and increments <code>TIMA</code> , overflow included. Both behaviours are required by the mooneye <code>div_write</code> and <code>div_trigger</code> tests.
<code>TIMA (\0xFF05)</code>	Read and write directly. Writing cancels a pending reload.
<code>TMA (\0xFF06)</code>	Read and write directly; the value is latched into <code>TIMA</code> when the pending reload fires.
<code>TAC (\0xFF07)</code>	Only bits 0–2 are writable. Reads return <code>\tac 0xF8</code> , so the four unused bits read as one.

→ `internal\_counter` starts at `\0xABCC` and advances one step per T-cycle, so `gb\_timer\_step(timer,n)` is the *only* thing that drives it. Calling the PPU or the APU directly, without the timer, produces a machine whose `\DIV` does not move.

9 The PPU

The PPU is a scanline renderer, not a pixel-by-pixel one. It advances a dot counter, changes modes at fixed thresholds, and draws a whole scanline into the frame buffer at a single moment — the mode 3 to mode 0 transition. There is no FIFO, no mid-scanline fetch and no per-pixel timing.

9.1 Registers

Address	Name	Bits and behaviour
\0xFF40	\LCDC	Bit 7 LCD enable, 6 window map, 5 window enable, 4 tile data source, 3 tile map source, 2 sprite size, 1 sprite enable, 0 BG/window enable.
\0xFF41	\STAT	Bits 1–0 are the mode (hardware-owned), bit 2 the \LYC=L Y coincidence flag, bits 3–5 the mode-interrupt enables, bit 6 the \LYC=LY interrupt enable. Writes keep only bits 3–6; reads additionally return \0x80.
\0xFF42	\SCY	Background scroll Y
\0xFF43	\SCX	Background scroll X
\0xFF44	\LY	Current scanline; read-only, writes ignored
\0xFF45	\LYC	The scanline compared against \LY
\0xFF47	\BGP	Two bits per shade for the background
\0xFF48	\OBP0	As above for sprites using palette 0
\0xFF49	\OBP1	As above for sprites using palette 1
\0xFF4A	\WY	First scanline on which the window is visible
\0xFF4B	\WX	Window left edge, in the hardware's \X+7 encoding

Registers outside \0xFF40--0xFF4B read as \0xFF. In particular the CGB palette registers \0xFF68 and \0xFF69 are not decoded and fall into the bus's shadow array, so they read back as whatever was last written there — zero, initially.

9.2 Mode state machine

`gb_ppu_step` receives a dot count, adds it to `\ppu->dots`, and checks the mode stored in the low two bits of `STAT`:

Mode	Name	Dots	Transition at
\2	OAM search	160	\dots =160 → mode 3
\3	Transfer	344 more	\dots =504 → draw, mode 0
\0	H-blank	408 more	\dots =912 → next line

Mode	Name	Dots	Transition at
\1	V-blank	912 per line	\ly 144–153, then wrap to 0

One scanline is therefore 912 dots, and a whole frame is \144*912 dots of active lines plus ten blanking lines. The drawing itself happens inside the mode 3 to mode 0 transition, in this order: background, window, sprites.

Two events are raised by the state machine:

- Entering mode 1 sets `\frame_ready` and `\vblank_interrupt`. `\frame_ready` is what tells `gb\_loop` to call `\render`.
- Every mode transition consults the mode-interrupt enables, and every line boundary consults `check\_lyc\_stat\_interrupt`, which maintains the coincidence flag and raises `\stat_interrupt` when bit 6 is set.

! Both interrupt flags are *booleans*, *not bits* in `\STAT`. They are latched into `\IF` by `gb\_handle\_interrupts` at the next instruction boundary and cleared there. A game that clears `\STAT`'s enable bits after the mode has already been entered will not see the interrupt, which differs from hardware but only matters for code that races the transition.

9.3 Turning the LCD on and off

Writing `\LCDC` bit 7 from 1 to 0 or from 0 to 1 resets the PPU:

- *Off*: `\ly=0`, `\dots=0`, mode forced to 0 (H-blank), `\first_line_after_enable` cleared. `gb\_ppu\_step` returns immediately from then on, so the display is frozen on the last rendered frame.
- *On*: the same three writes, plus `\first_line_after_enable=true`, which produces a shortened first line of 900 dots. At `\dots=900` the PPU jumps straight to line 1 in mode 2.

The 900-dot first line exists because the bl'rg `oam\_bug` composite reads `\LY` at roughly dot 896 and again at dot 904 and expects to see 0 and then 1. Getting that wrong is one of the ways the suite fails.

9.4 Rendering a scanline

`ppu_render_bg` walks all 160 screen columns and, for each one, computes the source address and writes one pixel:

```

line_y  = ly + SCY;
line_x  = x  + SCX;
tile_row = (line_y / 8) * 32;
tile_col = line_x / 8;
map_addr = tile_map_base + tile_row + tile_col;
tile     = bus_read(map_addr);
tile_addr = tile_data_base + tile*16 + (line_y % 8)*2;
bit      = 7 - (line_x % 8);
color    = ((plane2 >> bit & 1) << 1) | (plane1 >> bit & 1);
buffer[ly][x] = palette(BGP, color);

```

Both addressing modes of `\LCDC` bit 4 are implemented: unsigned from `\0x8000` and signed from `\0x9000` with the tile index read as a `int8_t`. Every fetch goes through `gb\_bus\_read`, so the renderer sees the

same memory the CPU does — including mapper effects on a game that (wrongly) puts tile data in ROM.

`ppu_render_window` is the same loop with `\line_y=ly-WY`, `\line_x=x-(WX-7)`, the LCD C bit 6 map, and a skip of every column left of the window's edge. The window has no internal line counter: its position within the map is recomputed from `\LY-WY` every line, which is why `\WY` must not change mid-frame if you want hardware-identical results.

`ppu_render_sprites` collects at most ten sprites whose 8 or 16 pixel band contains the current line, then draws them in reverse collection order so that the lowest OAM index ends up on top:

OAM byte	Meaning
0	Screen Y, biased by <code>\-16</code> ; a sprite spans <code>\ly</code> in <code>\[y-16,y-16+height)</code>
1	Screen X, biased by <code>\-8</code>
2	Tile index; bit 0 is ignored in 8×16 mode
3	Bit 7 BG priority, bit 6 Y flip, bit 5 X flip, bit 4 palette

Colour index 0 is transparent, so sprites never erase the background. Only the first ten matching sprites are drawn, which is the hardware's per-line limit rather than a choice.

! The BG-priority bit (bit 7 of OAM byte 3) is parsed but not applied: `ppu_render_sprites` reads it into `\bg_over` and then, as the in-code comment says, cannot tell whether the pixel underneath is background colour 0 without keeping the background colour indices around. Sprites therefore always draw over the background, and over each other, in OAM order. This is visible in games that put a sprite behind a background tile and expect it to disappear.

9.5 Palettes

There is one palette conversion routine for the whole system, `ppu_get_color(palette\_reg,color\_index)`, and one fixed four-entry table:

Shade	Colour	Value
0	white	<code>\0xFFFFFFFF</code>
1	light grey	<code>\0xFFAAAAAA</code>
2	dark grey	<code>\0xFF555555</code>
3	black	<code>\0xFF000000</code>

The shade is `\(palette_reg>>(color_index*2))&3`, so the DMG palette registers are honoured exactly; only the CGB's per-tile palettes are missing. Because the frame buffer is ARGB8888 with an opaque alpha, the terminal and WASM front ends can read the pixel as a plain `uint32_t` and shift it, and the ESP32 front end can convert it to RGB565 without knowing anything about the Game Boy.

9.6 What the PPU does not do

The following are absent, and are the reason several bl'rg and SameSuite tests fail:

- No per-pixel fetch timing, no FIFO, and no per-cycle object priority resolution.
- No blocking of VRAM access during modes 2 and 3, or of OAM during modes 2 and 3, or of writes to `\LY`.
- No `\STAT` blocking flags (the bit 7 “STAT blocking” bit), so a write to `\STAT` that hardware would delay takes effect at once.
- No DMG OAM scan bug. Sprites are read straight out of OAM at draw time, whereas hardware latches them during mode 2 and can be corrupted by writes in the meantime.
- No CGB: no palette RAM, no tile attribute byte, no VRAM banking, no separate window line counter, no `\BGPI`/`\BGPD`.

See section 16 for the full list and section 14 for how each gap shows up in the suites.

10 The APU

The APU synthesises all four channels at the emulated sample rate and hands the result to the host through a lock-free ring buffer. This is one of the more complete parts of the emulator; the gaps that remain are listed at the end of the section.

10.1 Registers

All forty-eight sound registers live in one array, `apu->regs[0x30]`, indexed by `\addr-0xFF10`, which is also how wave RAM is stored: the sixteen bytes at `\0xFF30-\0xFF3F` occupy `\regs[0x20]-\regs[0x2F]`.

Offset	Register	Role
\0x00	\NR10	Sweep period, negate, shift for channel 1
\0x01	\NR11	Duty and length load for channel 1
\0x02	\NR12	Envelope and DAC enable for channel 1
\0x03	\NR13	Frequency low byte, channel 1
\0x04	\NR14	Frequency high bits, length enable, trigger
\0x05-\0x09	\NR21-\NR24	As above for channel 2
\0x0A-\0x0E	\NR30-\NR34	DAC enable, sample volume, frequency, length enable, trigger for channel 3
\0x0F	\NR41	Length load for channel 4
\0x10	\NR42	Envelope and DAC enable for channel 4
\0x11	\NR43	Shift clock, width mode, divisor
\0x12	\NR44	Length enable and trigger
\0x14	\NR50	Master volume, left (bits 6-4) and right (2-0)
\0x15	\NR51	Panning: bits 0-3 right, bits 4-7 left
\0x16	\NR52	Master power (bit 7) and the four channel-on flags (bits 0-3)
\0x20-\0x2F	wave RAM	32 four-bit samples

Reads apply a per-register mask so that unused bits read as one, exactly as on hardware; NR52 is special-cased to synthesise the channel flags from `\ch_on[]`, and wave RAM is returned unmasked.

10.2 Power

NR52 bit 7 is `\apu->power`.

Powering off : Clears `\NR10-\NR51`, silences and disables every channel, disables every length counter, zeroes the frequency timers, the duty positions, the wave position, the frame sequencer and the LFSR, and reloads the LFSR with `\0x7FFF`. On a CGB cart the length counters are zeroed as well; on a DMG they are left alone.

Powering on : Sets `\power` and sets `\frame_div` to 1024, which places the sequencer in the second half of its period — the documented behaviour for a power-on during a frame.

While powered off : `\NR10`–`\NR51` are read-only. On a DMG cart the one exception is that a write to `\NRx1` still updates the length counter, which is what the length tests check. Wave RAM remains writable in both cases.

10.3 The frame sequencer

`apu->frame\_div` accumulates dots and steps every 2048 of them, i.e. at 512Hz in normal speed and 1024Hz in double speed. The eight steps do the following:

Step	Action
0, 2, 4, 6	Clock the length counters of all four channels; a counter reaching zero switches its channel off
2, 6	Step the sweep unit of channel 1
7	Step the volume envelopes of channels 1, 2 and 4

Why 2048 dots: The APU is stepped in dots, and the sequencer is defined in terms of the CPU frequency. 8192 dots — 2048 T-cycles at normal speed — is one 512Hz period, so the divisor is correct in both speed modes without any scaling.

10.4 Frequency timers

Each channel has a countdown that is decremented by the dot count every step; while it is non-positive the channel advances and the timer is reloaded. The reload value is `(2048-frequency)*4` for the two square channels and the wave channel, where `frequency` is the eleven-bit value assembled from `\NRx3` and the low three bits of `\NRx4`. For the noise channel it is four times one of eight divisors (`\8,16,32,48,64,80,96,112`) taken from the low three bits of `\NR43`.

What advances on each expiry:

- *Channel 1 and 2:* `\duty_pos` steps modulo 8.
- *Channel 3:* `\wave_pos` steps modulo 32.
- *Channel 4:* the 15-bit LFSR shifts right, feeding bit 0 into bit 14. With `\NR43` bit 3 set, bit 6 receives the XOR feedback instead of bit 1, which is the 7-bit width mode.

Duty patterns are the four standard constants — `\0x01`, `\0x81`, `\0x87`, `\0x7E` — indexed by bits 6–7 of `\NRx1`. The output of a square channel is its current volume when the selected bit of the duty pattern is set and zero otherwise.

10.5 Volume envelopes

`vol[3]`, `\env_period[3]`, `\env_counter[3]` and `\env_dir[3]` cover channels 1, 2 and 4; the wave channel has no envelope. At step 7 each channel with a non-zero period decrements its counter and, when that counter expires, reloads it and moves the volume one step up (direction set) or down (direction clear), clamped to 0–15. A triggering event reloads the volume from the high nibble of `\NRx2` and resets the counter.

! The envelope parameters are read out of `\NRx2` only on trigger, and `\env_period` and `\env_dir` are never updated from a later write to `\NR`

x2. Hardware applies them immediately. In practice a game writes NRx 2 once and then triggers, so this is only visible to tests that rewrite the envelope mid-note.

10.6 Sweep

Channel 1's frequency is shadowed in `\sweep_freq`. At sequencer steps 2 and 6 the counter is decremented, and when it expires the frequency is recomputed as `\sweep_freq` plus or minus `\(sweep_freq>>shift)`, where the sign is `\sweep_negate`. If the result exceeds 2047 the channel is switched off and the sweep disabled; otherwise the new value is written back into `\NR13` and `\NR14` and the shadow is recalculated. A second overflow check runs after the rewrite.

A trigger reloads the shadow from `\NR13/\NR14`, sets the counter from `\NR10` bits 4–6, and — if the shift is non-zero — performs the overflow calculation immediately, which is how the “channel off on trigger when the sweep overflows” case is handled. Writing `\NR10` with the negate bit cleared after negation has been used switches the channel off, via `\sweep_negate_used`.

10.7 Length counters

Writes to the four `\NRx1` registers load the counters: `\64-(value&0x3F)` for the square and noise channels and `\256-value` for the wave channel. Bit 6 of each `\NRx4` is the length enable. On trigger, a counter that is currently zero is unfrozen at 64 (or 256), and if the length enable is set and the sequencer is in the first half of its period, an extra length clock is applied immediately. The same extra clock is applied when the length enable bit goes from 0 to 1 during that half-period.

10.8 Triggers and DAC

A trigger (bit 7 of `\NRx4`) switches the channel on, reloads its frequency timer, resets the duty position or the wave position or the LFSR, and reloads the envelope. Immediately afterwards the DAC is checked: channels 1, 2 and 4 need `\(NRx2&0xF8)!=0` and channel 3 needs `\NR30` bit 7, so a trigger with the DAC disabled silently leaves the channel off. Conversely, writing a zero upper five bits to `\NRx2` switches the channel off without a trigger.

10.9 Mixing and output

Every `\GB_CPU_FREQ/\GB_APU_SAMPLE_RATE` dots — 190 at 44.1 kHz — the four channels are sampled:

1. Each channel is normalised to `\[0,1]`. The wave channel reads the sample nibble selected by `\wave_pos`, where the high nibble of a wave RAM byte comes first, and shifts it right by `\NR30` bits 5–4 (0 mutes, 1 is full scale, 2 halves it, 3 quarters it).
2. `\NR51` routes each channel to the left and/or the right bus.
3. `\NR50` scales each side by its three-bit volume divided by 7.
4. The two sides are added and divided by four — the code divides by 8, which leaves the result at roughly half scale — clamped to `\[-1,1]` and pushed onto the ring buffer.

The buffer is a 4096-entry single-producer single-consumer ring of floats with `\buf_write` and `\buf_read` marked `\volatile`. The producer is `gb\_apu\_step` on the emulation thread; the consumer is `gb\_apu\_buf\_pop`, which the SDL audio callback and the ESP32 audio task call. If the consumer falls behind, the producer simply drops samples — the push is skipped when the next index would collide with the read index, rather than overwriting unread audio.

10.10 Power-on defaults

`gb_apu_init` clears everything and then sets three registers that matter: `\NR52` to `\0xF1` (powered, all four channels reported on), `\NR11` to `\0xFF` (50% duty, full length) and `\NR51` to `\0xFF` (everything to both sides). The LFSR starts at `\0x7FFF` and wave RAM at zero.

10.11 What the APU does not do

- Envelope parameters are not latched from `\NRx2` on a plain write (see above).
- Wave RAM is not blocked while channel 3 is enabled, so a test that checks the read-back lock will fail.
- The output level is about half of what a real DMG produces, and the `\NR50` divide-by-zero case — both volumes zero — produces silence rather than the 50%-mute behaviour of real hardware.
- There is no high-pass filter, so the DC offset that gives the DMG its characteristic thin sound is absent.
- Length-counter reload and trigger timing are modelled in units of the frame sequencer rather than at cycle granularity, so the cycle-exact `blargg dm\_sound` tests still fail even where the audible result is right.

! The `TODO` file claims that `src/apu.c` is “minimal (length counters + `\NR52` on/off only)”. That is stale — the file now contains the full frame sequencer, sweep, envelope, wave and LFSR synthesis. Treat the `TODO` entries about the APU as a record of an older state of the code, and re-run the suites before acting on them.

11 Persistence

Two kinds of file are written next to the ROM, both named by appending an extension to the ROM path. Neither uses a directory: `gbemu.save` and `gbemu.state` appear next to the ROM, and if the ROM directory is not writable both operations fail with a message rather than falling back to a temporary location.

11.1 Battery saves

`gb\_battery\_save` writes `\<rompath>.sav`, but only for cartridge types that declare a battery-backed RAM:

Type	Cartridge	Payload
\0x03	MBC1 + RAM + battery	SRAM
\0x06	MBC2	512 bytes of 4-bit RAM
\0x09	ROM + RAM + battery	SRAM
\0x0F	MBC3 + timer + battery	SRAM + 5 RTC bytes
\0x10	MBC3 + timer + RAM + battery	SRAM + 5 RTC bytes
\0x13	MBC3 + RAM + battery	SRAM
\0x1B	MBC5 + RAM + battery	SRAM
\0x1E	MBC5 + rumble + RAM + battery	SRAM
other	none	nothing written

The SRAM payload is always the full `\sram.size` the loader allocated, not the size the header asked for — so the file is the 8 KiB minimum for small carts. For MBC2 it is exactly 512 bytes, the size of `\mbc2.ram`.

MBC3 timer carts append five bytes in the layout every other emulator uses: seconds, minutes, hours, the low byte of the day counter, and a high byte whose bit 0 is the ninth day bit, bit 6 the halt flag and bit 7 the carry flag. `\rtc.pack` and `\rtc.unpack` are the only places that conversion exists.

! Because the save covers the whole allocation, a `\.sav` written by one build is only readable by another build of the same emulator, and only for the same cartridge — the file has no header. It is, however, the layout other emulators use, so it is portable between emulators.

→ `gb\_battery\_load` reads the same file at start-up, prints `\[INFO] Batterysaveloaded:<path>` on success, and reports `\[ERR] Batterysaveistruncated:<path>` if the payload is short. A missing file is not an error: it simply returns `\false` and the game starts with a fresh save.

11.2 Save states

`gb\_save\_state` writes `\<rompath>.state` as a flat little-endian dump preceded by a 16-byte header:

```
typedef struct {
    char    magic[4];      // "EMU1"
    uint32_t emu_version;   // GB_VERSION_HEX
```

```

uint32_t rom_hash;      // FNV-1a over the padded ROM image
uint32_t sram_size;     // SRAM allocation at capture time
} StateHeader;

```

The payload follows in a fixed order, and the order is part of the file format:

1. the whole `\gb_cpu` struct
2. the whole `\gb_timer` struct
3. `\gb_ppu` up to `\offsetof(gb_ppu, frame_buffer)` — the registers and internal state, but *not* the 92 KiB of pixels
4. `\gb_apu` up to `\offsetof(gb_apu, audio_buf)` — the synthesis state, but not the ring buffer or its indices
5. the bus, field by field: the four memory arrays, `\io`, `\ie`, the whole DMA state, `\double_speed`, both joypad bytes, the mapper type and every mapper's registers (including the full `\mbc2_ram`), and finally the SRAM contents

Field-wise on purpose: The bus is written field by field rather than with one `\fwrite(bus, sizeof*bus)` so that the pointers `\rom`, `\sram`, `\timer`, `\ppu` and `\apu` — and the `\sram_dirty` hint — are automatically excluded. Two macros, `\W(field)` and `\R(field)`, do the work and are `\#undef`'d at the end of the file.

Two fields are deliberately not saved, and both are recreated or irrelevant afterwards:

- The `\apu` ring buffer indices are reset to zero after a load, so the audio callback cannot consume samples that belonged to the moment of capture.
- `\bus->joypad_interrupt` is not saved. It is a one-instruction edge, and the game will re-raise it if a button is still held.

11.3 Validation

`gb\_load\_state` refuses a state that does not match the running machine, and says why:

Check	Message on failure
File exists	<code>\[ERR]Nosavestatefound:<path></code>
Magic is <code>\EMU1</code>	<code>\[ERR]Notasavestate:<path></code>
<code>\emu_version==GB_VERSION_HEX</code>	<code>\[ERR]Savestategb_versionmismatch:file<n>,emulator<m></code>
<code>\rom_hash==fnv1a(rom, rom_size)</code>	<code>\[ERR]SavestatewascapturedfromadifferentROM</code>
<code>\sram_size</code> matches	<code>\[ERR]SavestateSRAMsizemismatch:...</code>
Payload fully readable	<code>\[ERR]Savestateistruncated:<path></code>

Hash choice: The ROM fingerprint is FNV-1a, a 32-bit multiply-and-xor with no multiplication table and no dependencies, computed over the whole *padded* buffer — including the zero fill above the file — and therefore after the interrupt-vector rewrite of section 6.4. Two builds of the emulator agree

only when `\GB_VERSION_HEX` agrees, so a state written by a debug build cannot be loaded by a release build of the same source.

! Save states are raw struct dumps, so they are not portable across architectures, compilers or struct layouts. They are a within-session convenience — suspend and resume — not an interchange format.

11.4 When things are written

Moment	Action
Start-up, before the front end is created	<code>gb_battery_load</code> for the ROM path
<code>\F5</code> or <code>\START+SELEC</code> T	<code>gb_save_state</code>
<code>\F9</code> or <code>\START+SELEC</code> T	<code>gb_load_state</code>
<code>gb_loop</code> returns	<code>gb_battery_save</code> , then <code>\fe->destroy</code>

A session that ends by killing the process rather than by closing the window loses both the battery save and any save state made since the last clean exit.

12 Front ends

Every front end implements the same four-function vtable and returns a `\gb_frontend*` from a `\frontend_*.create` constructor. Five of them exist; which one is compiled is decided at build time (section 2) or by the host application.

File	Constructor	Purpose
<code>frontend/sdl.c</code>	<code>\frontend_sd l_create(gb_ apu*)</code>	Window, texture, audio device, keyboard. The default build.
<code>frontend/terminal.c</code>	<code>\frontend_te rminal_creat e()</code>	ANSI true-colour output with half-block scaling, raw-mode keyboard.
<code>frontend/headless.c</code>	<code>\frontend_he adless_creat e()</code>	No output, no input, never terminates. The conformance harness.
<code>frontend/wasm.c</code>	<code>\frontend_wa sm_create()</code>	Blits the frame buffer into a canvas through an <code>\EM_JS</code> function.
<code>frontend/esp32.c</code>	<code>\frontend_es p32_create(g b_apu*)</code>	SPI ST7789/ST7735 panel, GPIO buttons, optional I2S DAC, on ESP-IDF 5.

12.1 The SDL front end

`sdl_init` initialises SDL's video and audio subsystems, then creates a window of `\160*scale` by `\144*scale` pixels (scale from `EMU\_SCALE`, default 4, clamped to at least 1), a streaming renderer, and an `\SDL_PIXELFORMAT_ARGB8888` texture — which is why the frame buffer is ARGB and not RGBA.

Audio uses `\SDL_OpenAudioDeviceStream` in `\SDL_AUDIO_F32` mono at `\GB_APU_SAMPLE_RATE`. The callback asks for `\additional_amount` bytes, fills them from `gb\_apu\_buf\_pop` in chunks of at most 1024 samples, and pushes them into the stream. If the emulator has not produced a sample the pop returns `\0.0f`, so a stalled emulator produces silence rather than stutter.

`sdl_poll_events` drains the queue and maintains the joypad bytes with read-modify-write operations, so two keys mapped to different bits never clobber each other. Both `\SDL_EVENT_KEY_DOWN` and `\SDL_EVENT_KEY_UP` are handled, which is why key repeat cannot accumulate. `\F5` and `\F9` fire only on the down transition.

One quirk: `joypad\_interrupt` is raised for *any* key press, including keys that are not mapped to a button. A game that enables the joypad interrupt and uses interrupts as a timing source will see extra requests.

12.2 The terminal front end

`frontend/terminal.c` needs no graphics library at all. It renders by printing one line of terminal rows per *two* Game Boy scanlines, using the half-block character `\U+2580` with a true-colour foreground for the upper pixel and background for the lower one:

```

for (y = 0; y < height - 1; y += 2)
    for (x = 0; x < width; x++)
        printf("\033[38;2;%d;%d;%dm\033[48;2;%d;%d;%dm\u2580", ...top, ...bottom);

```

The row buffer is 8192 bytes, sized for 160 pixels of escape sequences plus a reset and a newline. 143 rows are emitted, since the last scanline has no partner.

`term_init` puts the terminal in raw mode (`\ECHO`, `\ICANON` and `\IEXTEN` cleared, `\VMIN` and `\VTIME` zero) so that key presses arrive without a newline, hides the cursor, clears the screen and disables line wrapping. It also installs a `\SIGINT` handler that only sets a flag, because the loop must leave `gb_loop` before the terminal can be restored. `term_destroy` re-enables the cursor and wrapping and puts the original `\termios` back.

Keys are read with a single non-blocking `\read` of up to 64 bytes and decoded inline:

Input	Button	Note
<code>\ESC[A/B/C/D</code>	Up / Down / Right / Left	Arrow escape sequences; the sequence is consumed from the same buffer
<code>\z</code> , <code>\x</code>	A, B	
<code>\0x7F</code>	Select	The backspace byte
<code>\LF</code> or <code>\CR</code>	Start	
<code>\q</code> , <code>\Q</code>	—	Quit
bare <code>\ESC</code>	—	Quit
<code>\Ctrl-C</code>	—	Quit, via the signal flag

! Because there is no key-repeat protocol in a raw terminal, each press is treated as five frames long (`\KEY_HOLD_FRAMES`), which is about 83 ms, and released afterwards. A held arrow key therefore has to be pressed repeatedly. For the same reason `\term_poll_events` raises `\joypad_interrupt` unconditionally on every poll, so the joypad interrupt fires sixty times a second whether or not anything was pressed.

12.3 The headless front end

Three functions that do nothing and one that frees a `\NULL`. It never clears the `\running` flag, so a headless process runs until it is killed or signalled. That is exactly what `scripts/test` depends on: each ROM is run under `\gtimeout` and the harness reads its standard output. Any front end that returned `\false` from `\poll_events` would end the run immediately and break the harness.

12.4 The WebAssembly front end

`frontend/wasm.c` contains one `\EM_JS` function that copies the frame buffer into a `\ImageData` and paints it:

```

EM_JS(void, js_render_to_canvas, (const void* buffer, int w, int h), {
    var buf32 = new Uint32Array(Module.HEAPU8.buffer, buffer, w * h);
    for (var i = 0; i < w * h; i++) {
        data[i*4] = (buf32[i] >> 16) & 0xFF;    // ARGB -> RGBA
        data[i*4 + 3] = 0xFF;
    }
});

```

```

    }
    ctx.putImageData(imageData, 0, 0);
  });

```

The red and green channels are assigned inside the loop but are always `\0` for the current palette, since the emulator writes `\0xFF000000` black and `\0xFFFFFFFF` white. `\init` only sets the canvas element's `\width` and `\height` attributes; `\poll_events` is empty, because input arrives from JavaScript (see section 13.6).

12.5 The ESP32 front end

`frontend/esp32.c` is not built by `\make`; it is meant to be compiled inside an external ESP-IDF (5.0 or newer) project together with the core. Every piece of hardware configuration is a macro with a default, so the file can be dropped into a project for a different board without edits:

Group	Macros (defaults)
SPI	<code>\EMU_LCD_SPI_HOST</code> (<code>\SPI2_HOST</code>), <code>\EMU_LCD_PIN_SCLK</code> (18), <code>\MOSI</code> (23), <code>\CS</code> (15), <code>\DC</code> (2), <code>\RST</code> (4), <code>\BL</code> (21), <code>\EMU_LCD_BL_ACTIVE</code> <code>LOW</code> (0)
Panel	<code>\EMU_LCD_PANEL_ST7789</code> (1), <code>\EMU_LCD_WIDTH</code> (240), <code>\HEIGHT</code> (240), <code>\EMU_LCD_OFF_X/\Y</code> (centred), <code>\EMU_LCD_SPI_CLOCK_HZ</code> (80 MHz), <code>\MIRROR_X/\Y</code> (<code>\false</code>), <code>\COLOR_INVERT</code> (0)
Buttons	<code>\EMU_BTN_UP</code> (32), <code>\DOWN</code> (33), <code>\LEFT</code> (25), <code>\RIGHT</code> (26), <code>\A</code> (27), <code>\B</code> (14), <code>\SELECT</code> (13), <code>\START</code> (19)
Hotkeys	<code>\EMU_HOTKEY_SAVE_FRAMES</code> (30), <code>\LOAD_FRAMES</code> (150)
Audio	<code>\EMU_ENABLE_AUDIO</code> (1), <code>\EMU_I2S_PIN_BCLK</code> (27), <code>\LRCK</code> (26), <code>\DOUT</code> (25), <code>\EMU_AUDIO_TASK_CORE</code> (1)

Rendering converts ARGB8888 to RGB565 with the bytes swapped, because SPI panels shift out the most significant byte first, and blits with `\esp_lcd_panel_draw_bitmap` into the DMA-capable buffer. The frame is centred in the panel using the offset macros.

Audio runs in its own pinned FreeRTOS task which pops `gb\_apu\_buf\_pop` in blocks of 256 samples, converts to 16-bit integers and writes them to an I2S standard-mode channel; the blocking write is what paces the task. If either the channel allocation or the initialisation fails, audio is disabled and the emulator still runs.

! The default GPIO assignments for the buttons and the I2S pins overlap — `\EMU_BTN_B` is 14 while `\EMU_I2S_PIN_DOUT` is 25 and `\EMU_BTN_A` is 27 while `\EMU_I2S_PIN_BCLK` is 27 — so a board that enables audio *and* the A/B buttons on the default pins will not work. Override at least the I2S pins. The `\START` default of 19 is deliberately not 16 or 17, which are taken by PSRAM on WROVER boards.

→ The `\START+SELECT` combination is the only input on a device with no keyboard: 30 consecutive frames (about half a second) saves, 150 frames

(about two and a half seconds) loads. Releasing either button resets the counter.

13 Embedding the core

The library exposes everything an application needs and nothing else; the question is only which parts of the start-up sequence an embedding application is responsible for.

13.1 The minimal session

This is the whole of a working program apart from the front end:

```
#include "gbemu.h"

gb_cpu   cpu;
gb_bus   bus;
gb_timer timer;
gb_ppu   ppu;
gb_apu   apu;

bus.timer = &timer;    // wire the bus to its components FIRST
bus.ppu   = &ppu;
bus.apu   = &apu;

gb_cpu_init(&cpu);
gb_timer_init(&timer);
gb_ppu_init(&ppu);
gb_apu_init(&apu);

if (!gb_bus_load_rom(&bus, "game.gb")) return 1;
gb_battery_load(&bus, "game.gb");

bus.io[0x0F] = 0x01;    // IF: V-blank request left by the boot ROM
bus.ie       = 0x00;
bus.joyypad_dpad = 0x0F; // all released
bus.joyypad_buttons = 0x0F;

gb_frontend* fe = my_frontend_create();
fe->init(fe, GB_SCREEN_WIDTH, GB_SCREEN_HEIGHT);
gb_loop(&cpu, &bus, &timer, &ppu, &apu, fe);
gb_battery_save(&bus, "game.gb");
fe->destroy(fe);
```

! Four details are not optional:

1. **Wire the pointers first.** `\bus->ppu` and friends are tested for `\N` ULL inside the bus, so a missing one is a silent no-op rather than a crash.
2. **Call the four `\init` functions.** `\gb_cpu_init` sets `\PC` and the registers, `\gb_ppu_init` the video registers and clears the frame buffer, `\gb_apu_init` the sound defaults, and `\gb_timer_init` seeds the divider with `\0xABCC`.
3. **Set the post-boot interrupt state.** Neither `\gb_cpu_init` nor `\gb_timer_init` touches `\IF`, `\IE` or the joyypad bytes; the host has to.
4. **Decide `\A` from the cartridge.** Both `apps/cli.c` and `apps/wasm.c` force `\cpu.a=0x01` when header byte `\0x0143` is neither `\0x80` nor `\0xC0`, because `\gb_cpu_init` leaves the CGB boot value `\0x11` in place. Skipping this leaves DMG games in CGB mode.

13.2 Running without a loop

`gb\_loop` is a convenience, not a requirement. It is a plain frame-budgeted loop and can be replaced entirely; the WASM host already

does exactly that (section 13.6). The minimum sequence for one frame is:

```
int dots = 0;
while (dots < GB_CYCLES_PER_FRAME) {
    int t = gb_cpu_step(&cpu, &bus);
    int scale = bus.double_speed ? 1 : 2;
    dots += t * scale;
    int it = gb_handle_interrupts(&cpu, &bus, &ppu, &timer);
    dots += it * scale;
}
if (ppu.frame_ready) { /* draw */ ppu.frame_ready = false; }
```

Any front-end-like policy — rendering, input, sleeping, audio draining — can then be woven in wherever it suits the host.

13.3 Hotkeys and save states

The application, not the front end, decides what a hotkey means. Both `apps/cli.c` and an embedding application pass a context pointer and a callback:

```
typedef struct {
    gb_cpu* cpu; gb_bus* bus; gb_timer* timer;
    gb_ppu* ppu; gb_apu* apu; const char* rom_path;
} EmuContext;

static void on_hotkey(void* userdata, gb_hotkey key) {
    EmuContext* ctx = userdata;
    switch (key) {
        case GB_HOTKEY_SAVE_STATE:
            gb_save_state(ctx->cpu, ctx->bus, ctx->timer, ctx->ppu, ctx->apu,
                          ctx->rom_path);
            break;
        case GB_HOTKEY_LOAD_STATE:
            gb_load_state(ctx->cpu, ctx->bus, ctx->timer, ctx->ppu, ctx->apu,
                          ctx->rom_path);
            break;
    }
}

fe->hotkey_ctx = &ctx;
fe->on_hotkey = on_hotkey;
```

A front end that has no use for them leaves `\on_hotkey \NULL`, which is what the terminal, headless and WASM front ends do.

13.4 Audio consumer

The APU never blocks and never calls into the host. It produces samples at `\GB_APU_SAMPLE_RATE` and drops them if nobody is reading, so the only requirement on a host is to call `gb\_apu\_buf\_pop` periodically:

```
float s = gb_apu_buf_pop(&apu); // 0.0f when the buffer is empty
```

It is safe to do this from another thread — the ring buffer is single-producer single-consumer — and both real front ends do. Nothing else in the core is thread-safe: `\gb_cpu_step`, `\gb_bus_read`, `gb\_bus\_write` and `\gb_save_state` must all be called from the emulation thread.

13.5 Linking

```
cc -Isrc my_game.c -L. -lgbemu $(pkg-config --cflags --libs sdl3) -o my_game
cc -Isrc -DGBEMU_STRIP_PREFIX my_game.c libgbemu.a -o my_game
```

The static archive has the front ends *out* of it — only `src/*.c` is archived — so an embedding application links its own front end, or none at all. That is why the three constructor prototypes in `apps/cli.c` are declared locally rather than included from a header: exactly one of them is defined in the link.

! There is no `\main` in the archive. `apps/cli.c` and `apps/wasm.c` are the only programs, and both are compiled directly rather than archived.

13.6 The WebAssembly host

The browser build inverts the usual structure: the C side is driven from JavaScript and has no input loop of its own.

Exported symbol	Meaning
<code>\wasm_load_rom</code>	Zeroes all five state objects, re-runs the <code>\init</code> functions and loads <code>\rom.gb</code> from the virtual file system. Safe to call again to reset the session.
<code>\wasm_set_key(key,pressed)</code>	Maps a browser <code>\keyCode</code> to a joypad bit: 37–40 for the D-pad, 90 (<code>\Z</code>) for A, 88 (<code>\X</code>) for B, 8 for Select and 13 for <code>\Enter</code> . Raises the joypad interrupt on press.
<code>\wasm_is_running</code>	Returns 1 while a ROM is loaded and 0 otherwise, so the shell can decide whether to show the picker.
<code>\main</code>	Creates the front end, loads <code>\rom.gb</code> if it exists, and installs the main loop with <code>\emscripten_set_main_loop</code> in unlimited-render mode.

`web/shell.html` is the shell: a canvas with `\id="screen"`, a ROM picker, and the keyboard handling that calls `\wasm_set_key`. Because the loop runs in the browser's `\requestAnimationFrame`, frame pacing is the browser's job and the `\EMU_NOSLEEP` logic is not compiled in.

14 Testing

Correctness is measured against three external ROM suites that are run against a headless build of the emulator. No unit tests exist: the project has one test entry point, `scripts/test`, and its assertions live inside the ROMs.

14.1 The suites

Suite	Source	What it checks
gb	<code>\tests\gb-test-roms</code>	Blargg's classic test ROMs, both as composites and as individual cases: <code>\instr_timing</code> , <code>\interrupt_time</code> , <code>\halt_bug</code> , <code>\cpu_instrs</code> , <code>\mem_timing</code> , <code>\mem_timing-2</code> , <code>\oam_bug</code> , <code>\dmg_sound</code> , <code>\cgb_sound</code>
samesuite	<code>\tests/SameSuite</code>	SameBoy's hardware-behaviour ROMs, one per behaviour: <code>\cpu</code> , <code>\dma</code> , <code>\ppu</code> , <code>\apu</code> , <code>\timer</code> , <code>\interrupt</code> , <code>\sgb</code> and so on
mooneye	<code>\tests\mooneyetestsuite</code>	The acceptance and emulator-only ROMs whose hardware suffix says they are expected to pass on a DMG

SameSuite ROMs are assembled with `rgbds` and mooneye ROMs with `wladx`; the script builds them if the tools are present, and skips the build if prebuilt ROMs already exist.

mooneye filtering: mooneye names carry a hardware suffix after the last hyphen — an empty suffix means “all models”, `\-GS` means Game Boy and Super Game Boy, `\-dmgABCmgb` names specific models, `\-sgb` is Super Game Boy only. The harness keeps a test only if there is no suffix, or the suffix contains `\dmg` or a capital `\G`. This is why a CGB-only test does not appear as a failure: it is never run.

14.2 Verdicts

Verdicts arrive over the emulated serial port, using the two interception paths of section 6.6.

SameSuite and mooneye : The ROM writes a result byte per iteration through `\SB/\SC`. Success is the Fibonacci register `\0305080D1522`; failure is six `\0x42` bytes. The script pipes the emulator's standard output through `\xd-p` and compares the hex string.

Blargg : The ROM prints human-readable lines through cartridge RAM at `\0xA004` and finishes with `\Passed` or `\Failed`. The script greps for `\passed` while requiring that `\failed` does *not* also appear, which distinguishes a partial composite report from a clean pass.

! The `\Passed` test runs before the `\Failed` test on purpose: the bl'rg composites print every individual result and a final verdict, and a ROM that prints both `\01:ok02:ok...Failed` must be scored as a failure.

14.3 Running the harness

```
./scripts/test          # all three suites
./scripts/test gb       # only the bl'rg ROMs
./scripts/test samesuite # only SameSuite
./scripts/test mooneye   # only mooneye
./scripts/test --help
```

The script sets two environment variables that matter: `\EMU_NOSLEEP=1` so that each ROM runs as fast as the CPU allows, and `\EMU_HEADLESS=1` so that no window is created. It then rebuilds the emulator:

```
make clean
make all type=RELEASE HEADLESS=1
```

! The rebuild is unconditional and it *overwrites* a debug build. If you are running the harness under a debugger, or have `libgbemu.a` linked into something else, build the emulator again afterwards — and remember that the release build has no sanitizers, so memory errors that the debug build would report here simply corrupt the picture.

Each ROM runs under `\gtimeout` with a suite-specific limit: 60 seconds for the bl'rg composites, 30 seconds for individual bl'rg ROMs, and 15 seconds for SameSuite and mooneye, which print their result within a few seconds and then halt forever.

14.4 Report files

Three reports are written, one per suite: `test-report-gb.txt`, `test-report-samesuite.txt` and `test-report-mooneye.txt`. Each test contributes one block:

```
TEST: [composite] instr_timing
File:  ../tests/gb-test-roms/instr_timing/instr_timing.gb
Lines: 4
Status: PASS
Detail: instr_timing; ; ; Passed;
---
```

`\Status` is one of six values:

Status	Meaning
\PASS	The expected signature was seen
\FAIL	The failure signature was seen, or the text verdict said <code>\Failed</code> , or nothing was printed at all (<code>\NOOUT</code>)
\NOOUT	The emulator exited without producing any output
\TIMEOUT	The emulator was killed having produced nothing
\PARTIAL	The emulator was killed having produced a prefix of the signature — the most useful status, since it says how far it got
\UNKNOWN	Some other output

A per-suite summary with the counts of each status is appended to each report.

14.5 Reading a failure

1. Look at `\Detail`. For a SameSuite or mooneye ROM it is the raw serial bytes; `\4242` means the ROM decided it had failed.
2. A `\PARTIAL` serial result is not a failure of the emulator — it means the ROM printed its result and then kept running, so it usually indicates that the test itself passed and the timeout was simply too short, or that the ROM halted before finishing its remaining iterations.
3. Check `\Lines` for the bl'rg composites: a composite that dies early prints far fewer lines than the last one in a passing run.
4. Cross-check the group name against section 16. Most failures have a documented cause, and re-running the suite after fixing one of them usually moves the failure elsewhere rather than removing it.

! `\tmp/blargg_raw.bin` accumulates every byte written to cartridge RAM at `\0xA004` and above, including non-printable ones, for the whole process. It is truncated when the first write happens, so run one ROM at a time if you want to inspect it. The file is not created by the ESP32 build.

14.6 What the suites do not cover

- No multi-frame determinism test: there is no test that runs the same ROM twice and compares the frame buffers.
- No fuzzing of the bus, so an access to an address the mapper model does not expect is caught only if a ROM happens to make it.
- No performance regression tracking. `\EMU_NOSLEEP=1` makes the harness run as fast as possible but nothing measures the rate.
- The bl'rg and mooneye suites are third-party; `\makeall` does not fetch them, and the script fails with an explicit message if a suite's ROMs are neither present nor buildable.

15 Extending the emulator

This chapter is about changing the code without breaking its two contracts: the cycle count an instruction reports, and the size and layout of the structs a save state dumps.

15.1 Adding an instruction

The generated table has to know about it first. Add the entry to `docs/dev/Opcodes.json` with its mnemonic, operands, size and cycle counts, then regenerate:

```
python3 scripts/generate_opcodes.py
```

The script reads the JSON, derives the `\OP_*` enum name from the mnemonic and operands using the rules in section 5.5, and rewrites `src/opcodes.h` from scratch. Editing `opcodes.h` by hand is pointless: the next regeneration discards it.

Then implement the body in the handler that already owns that category, and add the `\case` label:

```
case OP_SLA_A: {
    uint8_t carry = (cpu->a >> 7) & 1;
    cpu->a = cpu->a << 1;
    gb_flag_set(cpu, GB_FLAG_Z, cpu->a == 0);
    gb_flag_set(cpu, GB_FLAG_N, false);
    gb_flag_set(cpu, GB_FLAG_H, false);
    gb_flag_set(cpu, GB_FLAG_C, carry);
    break;
}
```

! Three things must all stay true, and nothing checks any of them:

1. **The number of machine cycles matches the table.** Count the `\gb_machine_tick(bus,4)` calls made on the execution path, including the ones inside `gb\_fetch8`, `gb\_fetch16` and the register-index helpers, and add the internal cycles. The total must equal the entry's cycle count divided by four.
2. **Flags are set explicitly.** Every arithmetic or logical instruction must write `\Z`, `\N` and `\H` even when the answer is “unchanged”: on hardware those flags are not preserved.
3. **\POPAF masks the low nibble.** If your instruction can affect `F` indirectly, check that path.

The bl'rg `\cpu_instrs` and SameSuite `\cpu` suites are the tests that will find a mistake here.

15.2 Adding a component

A new emulated device is a struct plus a `\step` function plus a call site in `gb\_machine\_tick`:

```
typedef struct gb_something { /* ... */ } gb_something;

void gb_something_init(gb_something* d);
void gb_something_step(gb_something* d, int dots);
uint8_t gb_something_read(const gb_something* d, uint16_t addr);
void gb_something_write(gb_something* d, uint16_t addr, uint8_t v);
```

```
// in gb_machine_tick
if (bus->something) gb_something_step(bus->something, t_cycles * scale);
```

! If you also want it in save states, add it to the `\W` and `\R` chains in `save.c` — in the same position on both sides, since the format is positional. And keep the large buffers at the *end* of the struct, because `gb\_save\_state` serialises a component with `fwrite(ppu, offsetof(gb_ppu, frame\_buffer), 1, f)` and a new member inserted before the buffer would silently be included.

Bus routing follows the existing pattern: test `\bus->something!=NULL` before delegating, and fall through to the `\io` shadow array otherwise, so a bus without the component still answers reads.

15.3 Adding a front end

Write four functions and one constructor, following `frontend/headless.c`, which is the whole of a working (if useless) front end:

```
static bool my_init (gb_frontend* fe, int w, int h);
static void my_render(gb_frontend* fe, const uint32_t* buf, int w, int h);
static void my_poll (gb_frontend* fe, gb_bus* bus, bool* running);
static void my_destroy(gb_frontend* fe);

gb_frontend* my_frontend_create(void) {
    gb_frontend* fe = calloc(1, sizeof(gb_frontend));
    fe->priv = my_private_state();
    fe->init = my_init;
    fe->render = my_render;
    fe->poll_events = my_poll;
    fe->destroy = my_destroy;
    return fe;
}
```

Rules that the loop depends on:

- `\init` must return `\false` rather than aborting; `\main` turns that into exit status 1.
- `\render` must not keep the buffer pointer. The core clears `\frame\_ready` immediately after the call and the pixels are overwritten on the next scanline.
- `\poll_events` is the only way to stop the machine, through the `\running` out-parameter. A front end that never clears it can only be terminated from outside — which is the headless design.
- `\destroy` must release what `\init` allocated. The core does not touch the vtable again.

Then add it to `make/common.mk` as another `\FRONTEND_SRC` case, and add the corresponding `\-D test` to `apps/cli.c`.

15.4 Adding an interrupt

1. Reserve the bit in `gb_bus`: a boolean next to the existing request flags is enough.
2. Set it where the condition is detected, e.g. at the end of a scanline in `gb\_ppu\_step`.
3. Latch it into `\IF` in `gb\_handle\_interrupts`. That function is the *only* place hardware flags become register bits, and the pending

mask there is `\if_reg&bus->ie&0x1F` — a new bit needs widening, or it will be invisible.

4. Add the vector case to the loop in the same function. The dispatch is a linear scan from bit 0, so the loop bound and the vector arithmetic (`\0x0040+8*bit`) both need updating.
5. Extend the save state if the flag is persistent.

! The serial interrupt (`\IF` bit 3) is the worked example of how not to do this: `\IF` bit 3 is reachable and dispatchable, but nothing ever raises it, because serial output is handled by interception instead. The vector at `\0x0058` exists and costs nothing until it is used.

15.5 Changing the timing model

Two constants govern everything, and both are derived from the clock rather than hard-coded:

Macro	Definition
<code>\GB_CPU_FREQ</code>	<code>\4194304</code> T-cycles per second, the DMG rate
<code>\GB_CYCLES_PER_FRAME</code>	<code>\GB_CPU_FREQ/60</code> , i.e. 69 905 T-cycles
<code>\GB_FRAME_TIME_NS</code>	<code>\1000000000L/60</code>

Note that `\GB_CYCLES_PER_FRAME` is expressed in T-cycles but is compared in `gb\_loop` against a running total of *dots*, so the two agree only because one T-cycle is two dots at normal speed. That is not an accident: it is what keeps the frame budget correct in CGB double-speed mode, where the scale flips to 1. Any change to the cycle handling has to preserve that relationship.

16 Known gaps

This chapter collects, in one place, the hardware behaviour the emulator does not implement. Each entry says what is missing and which suite notices. Most of them were found by running the suites, not by reading the Game Boy documentation.

16.1 CPU and timing

Gap	Effect
Atomic instructions	An instruction executes as a unit: its total cost is returned from the table, and its immediate reads and writes happen in handler order rather than at fixed dots. The bl'rg <code>\mem_timing</code> suites, which read a register at a precise dot inside <code>\LDHA,(a8)</code> , fail all their cases. See also <code>\ADDSP,e8/\LDHL,SP+e8</code> , whose internal cycles are modelled as plain increments.
<code>\EI</code> and <code>\HALT</code>	The interaction is not exact: <code>\ime_scheduled</code> is promoted at the top of <code>gb_cpu_step</code> , before the halted check, so a <code>\EI;HALT</code> sequence does not wake the CPU the way hardware does. SameSuite <code>\interrupt/ei_delay_halt</code> fails; the bl'rg <code>\halt_bug</code> composite passes.
<code>\STOP</code>	Performs the CGB speed switch and consumes its operand byte, then returns. It does not stop the CPU, halt the divider, mute the APU or reset the joypad matrix, all of which real hardware does.
<code>\DAA</code>	Implemented, but the C-64 (non-carry) case only ever clears <code>\C</code> when the addition overflowed, rather than preserving it.

16.2 Memory, DMA and the bus

Gap	Effect
No DMA register validation	The OAM DMA source page is not masked and not checked against <code>\0xFF00--\0xFF9F</code> . A page of <code>\0xFE</code> copies <code>\0xFF</code> bytes, because OAM reads are themselves blocked during the transfer.
No DMA restart timing edge	A <code>\0xFF46</code> write during the initial two-cycle delay, or while a restart is already pending, restarts immediately instead of after two cycles (section 6.7).
No CGB DMA	<code>\GDMA</code> and <code>\HDMA</code> at <code>\0xFF51--\0xFF55</code> are not implemented at all, so CGB games that use them hang. SameSuite <code>\dma/hdma_lcd.off</code> and friends fail.

Gap	Effect
Unmapped I/O reads	Fall through to the <code>\io</code> shadow array, which reads back whatever was last written — zero initially — instead of the <code>\xFF</code> real hardware returns.
Echo RAM at <code>\xFD00–\xFDFF</code>	Correctly aliases <code>\xC000–\xCFFF</code> , so games that probe <code>\xFFFE</code> in the echo region see HRAM contents rather than a fault.

16.3 Mappers

Gap	Effect
MBC3 RTC is not driven by a clock	The latch sequence advances a state machine but never copies the host time into the registers, and the counters only advance when a game writes them. A game that reads the clock sees whatever it found at load time. There is a <code>\TODO</code> marker for this.
Serial text output for MBC2 and MBC3	The <code>\xA004</code> interception is after the MBC2 and MBC3 write handlers, which return early, so text diagnostics from those mappers never reach standard output. Only the <code>\SB/\SC</code> path works for them.
MBC3 does not distinguish RAM banks 0–3 by size	The bank mask is <code>\ram_bank&(sram_banks-1)</code> , so a cart declaring 8 KiB of RAM has only one reachable bank and selecting bank 3 silently wraps.
No MBC6, MBC7, HuC1, camera or rumble	Cartridge types outside the four supported families are treated as ROM-only. A rumble cartridge runs, but the motor bit at <code>\x6000</code> is discarded.

16.4 PPU

See section 9.6 for the full list. In brief: no per-pixel timing, no VRAM/OAM access blocking, no `\STAT` blocking flag, no `\LY`-write side effects, no DMG OAM scan bug, sprite BG priority not applied, no CGB palette RAM or tile attributes, no VRAM banking.

16.5 APU

See section 10.11. In brief: envelope parameters not latched on write, no wave RAM locking, no high-pass filter, output level about half scale, and cycle-exact length/trigger timing is not modelled.

16.6 CGB

Missing	Consequence
Palette RAM (<code>\BGPI</code> / <code>\BGPD</code>)	Colour 0 of every tile uses the DMG greyscale table, so CGB games display in greyscale. SameSuite <code>\ppu/blocking_bgpi_increase</code> fails.

Everything else about the CGB — VRAM banking, tile attributes, the separate window line counter, `\OPRB`, HDMA, the boot ROM and the `\KEY1` management registers beyond bit 0 — is absent. `\double_speed` and `\STOP` do work, so a CGB game that only checks which mode it is in will behave correctly.

16.7 SGB

Nothing is implemented: no `\MLT_REQ`, no border palette, no attribute files, no joypad comms. Two SameSuite tests, `\sgb/command_mlt_req` and `\sgb/command_mlt_req_1_incrementing`, fail; the `TODO` file suggests either implementing the commands or filtering the tests out.

16.8 Front ends

- The SDL and terminal front ends raise `\joypad_interrupt` more eagerly than hardware: SDL does so for any key, the terminal for every poll.
- The terminal front end has no key repeat, so buttons are five frames long and must be re-pressed.
- The headless front end provides no way to end a session, and no way to observe anything.
- The ESP32 front end's default pin assignments conflict between the buttons and the I2S pins (section 12.5).

16.9 Stale documentation

! The `TODO` file at the repository root is out of date. Among other things it claims that `src/apu.c` contains only length counters and that `src/bus.c` has no DMA support — both of which have since been implemented. Read it as a record of what was missing when the entry was written, and confirm against the source before acting on an item.

A Opcode coverage

Of the 256 base opcode values, 244 are dispatched, one is the `\CB` prefix, and eleven are illegal and fatal:

Opcode	Status
<code>\0xCB</code>	The prefix byte; handled by <code>instr_cb</code> , never by a handler
<code>\0xD3</code> , <code>\0xDB</code> , <code>\0xDD</code>	Illegal
<code>\0xE3</code> , <code>\0xE4</code>	Illegal
<code>\0xEB</code> , <code>\0xEC</code> , <code>\0xED</code>	Illegal
<code>\0xF4</code>	Illegal
<code>\0xFC</code> , <code>\0xFD</code>	Illegal

The generated table prices all of them, including the illegal ones, so `get\_opcode\_cycles` has no hole in it. Reaching any of them prints `\Unhandledopcode:0xNNatPC:0xNNNN` and calls `\exit(1)`.

The eleven handlers own the remaining 244:

Handler	Opcodes	Category
<code>instr_8bit_arithm</code>	61	8-bit arithmetic and logic
<code>instr_load</code>	92	Loads and stack-relative loads
<code>instr_16bit_arithm</code>	13	16-bit arithmetic
<code>instr_bitwise</code>	28	Bitwise and compare
<code>instr_bit_flag</code>	0	Empty placeholder
<code>instr_bit_shift</code>	4	Accumulator rotates
<code>instr_jumps</code>	30	Jumps, calls, returns
<code>instr_carry</code>	2	Carry flag control
<code>instr_interrupts</code>	3	<code>\DI</code> , <code>\EI</code> , <code>\HALT</code>
<code>instr_stack</code>	8	<code>\PUSH</code> and <code>\POP</code>
<code>instr_misc</code>	3	<code>\DAA</code> , <code>\NOP</code> , <code>\STOP</code>
Total	244	

The `\CB` space is dispatched by field rather than by table, so all 256 values are covered: 8 rotates/shifts, 8 `\BIT`, 8 `\RES` and 8 `\SET` per register, including `\(HL)`.

B I/O register map

The registers the emulator decodes, with their destinations. Everything else in `\0xFF00--\0xFF7F` is a plain byte in `\bus->io`.

Address	Register	Handled by
<code>\0xFF00</code>	<code>\P1</code>	<code>\gb.bus_read/\write</code> , joystick matrix
<code>\0xFF01</code>	<code>\SB</code>	<code>\io</code> shadow; source for the serial print
<code>\0xFF02</code>	<code>\SC</code>	Special: writing <code>\0x81</code> prints <code>\SB</code>
<code>\0xFF04</code>	<code>\DIV</code>	<code>\gb.timer_read/\write</code>
<code>\0xFF05</code>	<code>\TIMA</code>	<code>\gb.timer_read/\write</code>
<code>\0xFF06</code>	<code>\TMA</code>	<code>\gb.timer_read/\write</code>
<code>\0xFF07</code>	<code>\TAC</code>	<code>\gb.timer_read/\write</code>
<code>\0xFF0F</code>	<code>\IF</code>	Special: reads OR <code>\0xE0</code> , writes mask <code>\0x1F</code>
<code>\0xFF10--\0xFF2F</code>	<code>\NR10--\NR51</code>	<code>\gb.apu_read/\write</code>
<code>\0xFF30--\0xFF3F</code>	wave RAM	<code>\gb.apu_read/\write</code> , unmasked reads
<code>\0xFF40</code>	<code>\LCDC</code>	<code>\gb.ppu_read/\write</code> , power transitions
<code>\0xFF41</code>	<code>\STAT</code>	<code>\gb.ppu_read/\write</code> , bits 3-6 only
<code>\0xFF42</code>	<code>\SCY</code>	<code>\gb.ppu_read/\write</code>
<code>\0xFF43</code>	<code>\SCX</code>	<code>\gb.ppu_read/\write</code>
<code>\0xFF44</code>	<code>\LY</code>	<code>\gb.ppu_read</code> ; writes ignored
<code>\0xFF45</code>	<code>\LYC</code>	<code>\gb.ppu_read/\write</code>
<code>\0xFF46</code>	<code>\DMA</code>	Special: starts an OAM transfer
<code>\0xFF47</code>	<code>\BGP</code>	<code>\gb.ppu_read/\write</code>
<code>\0xFF48</code>	<code>\OBP0</code>	<code>\gb.ppu_read/\write</code>
<code>\0xFF49</code>	<code>\OBP1</code>	<code>\gb.ppu_read/\write</code>
<code>\0xFF4A</code>	<code>\WY</code>	<code>\gb.ppu_read/\write</code>
<code>\0xFF4B</code>	<code>\WX</code>	<code>\gb.ppu_read/\write</code>
<code>\0xFF4D</code>	<code>\KEY1</code>	Special: bit 7 from <code>\double_speed</code> , bit 0 stored
<code>\0xFFFF</code>	<code>\IE</code>	<code>\bus->ie</code> , all eight bits

Memory regions that are decoded but not emulated in full:

Range	State
<code>\0xFE00--\0xFEAF</code>	OAM; reads return <code>\0xFF</code> and writes are dropped while a DMA transfer is copying
<code>\0xFE00--\0xFE0F</code>	Unusable; reads <code>\0xFF</code> , writes dropped
<code>\0xFF68, \0xFF69</code>	CGB palette; not decoded, lands in the shadow array
<code>\0xFF51--\0xFF55</code>	CGB DMA; not decoded

C Public API reference

The complete surface of `src/gbemu.h`, in declaration order.

C.1 CPU

Function	Purpose
<code>\gb_cpu_init</code>	Set <code>\PC</code> to <code>\0x0100</code> and the post-boot registers
<code>\gb_cpu_step</code>	Fetch, decode and execute one instruction; return its T-cycles, or 4 while halted
<code>\gb_cpu_dump_fd</code>	Print the register file to a <code>\FILE*</code>
<code>\gb_flag_set / \gb_flag_get</code>	Set or test one flag
<code>\gb_get_reg_by_index / \gb_set_reg_by_index</code>	Operand access by the 3-bit index, including the <code>\(HL)</code> machine cycle
<code>\gb_fetch8 / \gb_fetch16</code>	Operand fetch, one machine cycle per byte
<code>\gb_instr</code>	Decode and dispatch one opcode; returns its T-cycles

C.2 Bus

Function	Purpose
<code>\gb_bus_read / \gb_bus_write</code>	The whole memory map
<code>\gb_bus_tick</code>	Advance the OAM DMA by one machine cycle
<code>\gb_bus_load_rom</code>	Load and pad a ROM, size the SRAM, configure the mapper, sanitise the interrupt vectors; returns the byte count or 0
<code>\gb_machine_tick</code>	Advance DMA, timer, PPU and APU

C.3 Timer, PPU and APU

Function	Purpose
<code>\gb_timer_init / \gb_timer_step</code>	Initialise and advance the divider and timer
<code>\gb_timer_read / \gb_timer_write</code>	The four registers
<code>\gb_get_time_ns / \gb_sleep_ns</code>	Portable monotonic clock and sleep
<code>\gb_ppu_init / \gb_ppu_step</code>	Initialise and advance the scanline state machine
<code>\gb_ppu_read / \gb_ppu_write</code>	The video registers
<code>\gb_apu_init / \gb_apu_step</code>	Initialise and advance the synthesis

Function	Purpose
<code>\gb_apu_read / \gb_apu_write</code>	The sound registers
<code>\gb_apu_buf_pop</code>	The one function a host may call from another thread
<code>\gb_handle_interrupts</code>	Latch request flags into <code>\IF</code> , wake the CPU, dispatch; returns 0 or 20

C.4 Loop and persistence

Function	Purpose
<code>\gb_loop</code>	The frame-budgeted main loop
<code>\gb_battery_load / \gb_battery_save</code>	The <code>\.sav</code> file
<code>\gb_save_state / \gb_load_state</code>	The <code>\.state</code> file

→ `src/frontend.h` adds the `\gb_frontend` struct, the `\gb_hotkey` enum and `\gb_version`, which expand to the version string, `\0.1.0`, and `\GB_VERSION_HEX`, which expands to `\0x100`.

D Constants

Every compile-time constant in the public header, with the value it has in this version.

Macro	Value	Meaning
\GB_CPU_FREQ	\4194304	T-cycles per second
\GB_CYCLES_PER_FRAME	\69905	\GB_CPU_FREQ/60
\GB_FRAME_TIME_NS	\16666666	one second, divided by 60
\GB_SCREEN_WIDTH	\160	Frame buffer width
\GB_SCREEN_HEIGHT	\144	Frame buffer height
\GB_APU_BUF_SIZE	\4096	Ring buffer capacity, a power of two
\GB_APU_BUF_MASK	\4095	\SIZE-1, for wrapping
\GB_APU_SAMPLE_RATE	\44100	Output sample rate
\GB_VERSION_MAJOR	\0	
\GB_VERSION_MINOR	\1	
\GB_VERSION_PATCH	\0	
\GB_VERSION_HEX	\0x100	\0x00MMmmpp, written into save states

PPU modes and CPU flags:

Constant	Value	Meaning
\GB_PPU_MODE_HBLANK	\0	Mode 0, \STAT bits 1–0 \00
\GB_PPU_MODE_VBLANK	\1	Mode 1, \01
\GB_PPU_MODE_OAM	\2	Mode 2, \10
\GB_PPU_MODE_XFER	\3	Mode 3, \11
\GB_FLAG_Z	\0x80	Bit 7 of F
\GB_FLAG_N	\0x40	Bit 6
\GB_FLAG_H	\0x20	Bit 5
\GB_FLAG_C	\0x10	Bit 4

E Further reading

The specifications and test ROMs this emulator is written against. All of them are free and public.

E.1 Reference documentation

Source	Covers
<code>\gbdev.io/pandocs/PPU.html</code>	Mode timing, <code>\STAT</code> , <code>\LY</code> / <code>\LYC</code> , the background, window and sprite rules, and the CGB additions
<code>\gbdev.io/pandocs/Memory_Map.html</code>	The address map, the I/O register semantics and the DMA rules
<code>\gbdev.io/pandocs/Timer_and_Divider_Registers.html</code>	The divider bit select, the <code>\TIMA</code> overflow sequence and the <code>\DIV</code> -write side effects
<code>\gbdev.io/pandocs/Audio.html</code>	The four channels, the frame sequencer steps, the sweep overflow rules and the length-counter timing
<code>\gbdev.io/pandocs/Cartridge_Header.html</code>	Header bytes <code>\0x0143</code> , <code>\0x0147</code> , <code>\0x0148</code> , <code>\0x0149</code> and the Nintendo logo
<code>\gbdev.io/pandocs/Interrupt_Handling.html</code>	The <code>\IF</code> / <code>\IE</code> contract, the five vectors and the five machine cycles of dispatch
<code>\gbdev.io/pandocs/MBC1.html</code> , <code>\MBC2</code> , <code>\MBC3</code> , <code>\MBC5</code>	One page per implemented mapper

E.2 Test ROMs

Suite	Note
<code>\gb-test-roms</code>	Blargg's ROMs, vendored under <code>\tests/</code> . The most widely used functional suite; the source of most of the known gaps in section 16.
<code>\SameSuite</code>	SameBoy's suite, vendored under <code>\tests/</code> . One ROM per hardware behaviour, which makes it the best tool for localising a bug.
<code>\mooneye-test-suite</code>	The acceptance tests, which are the closest thing to a specification that can be executed. Several tests only pass on hardware; the harness filters by the suffix in the name.

E.3 Internal notes

Within the repository, `docs/dev/` holds the developer-facing material that is not part of this manual: the header-comment markers for `\tinydocs-cli`, the opcode JSON the generator consumes, and progress notes. `\makedocs` regenerates the API pages from the headers.

F Index

Index

- code generation, 2
- cycle accounting, 15
- cycle-driven, 2

- documentation, 2

- frontend
 - independence, 2

- gbemu, 2

- license, 5

- portability, 2

- STOP, 4

- unmapped I/O, 60